# Table of Contents


1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

# Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

# What is GEMM?

**Definition: Single-precision general matrix multiplication (SGEMM)**

$$C \leftarrow \alpha AB + \beta C$$

# What is GEMM?

**Definition: Single-precision general matrix multiplication (SGEMM)**

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

# What is GEMM?

## Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

## Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

# What is GEMM?

## Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

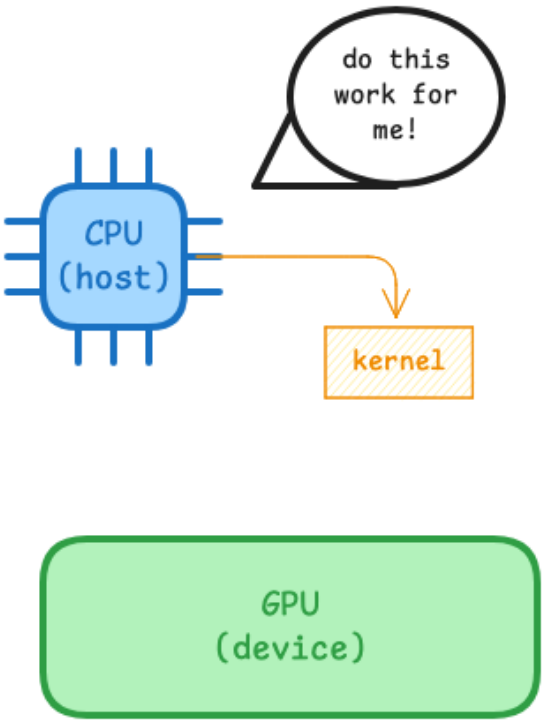
```
sgemm(
  int M, int N, int K,
  float alpha, float *A, float *B,
  float beta, float *C
)
```

$$\underbrace{A}_{M \times K} \quad \underbrace{B}_{K \times N} \quad \underbrace{C}_{M \times N}$$

- $M$: rows of $A$ and $C$
- $N$: columns of $B$ and $C$
- $K$: shared reduction dimension

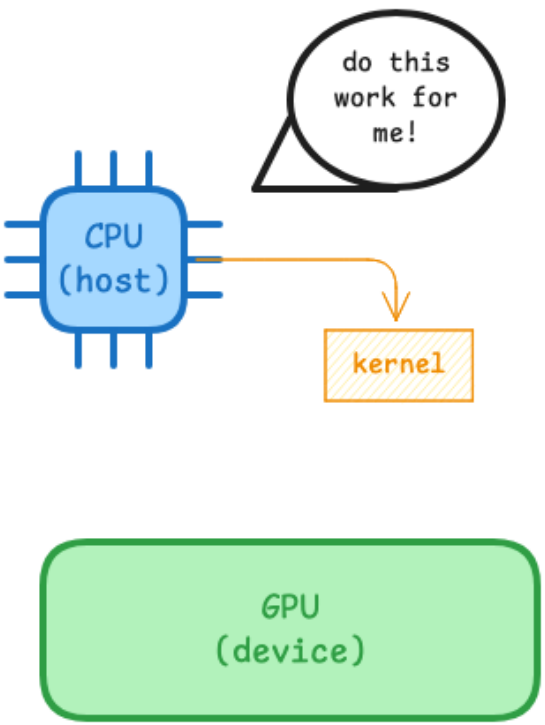
# CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.



# CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

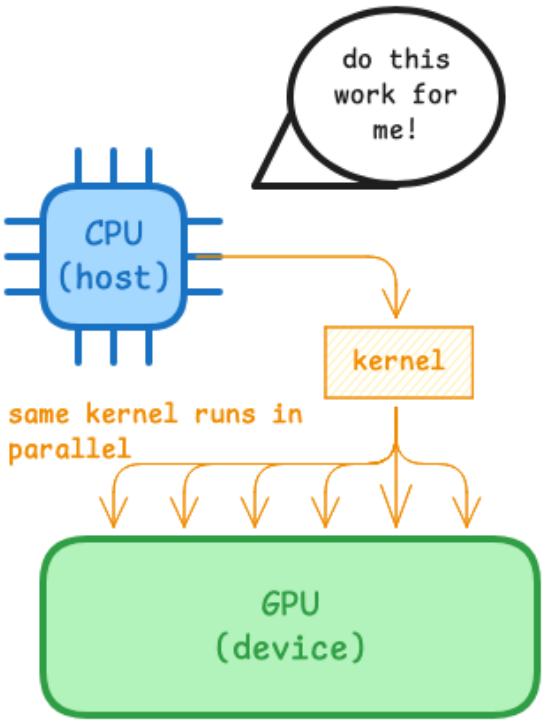
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

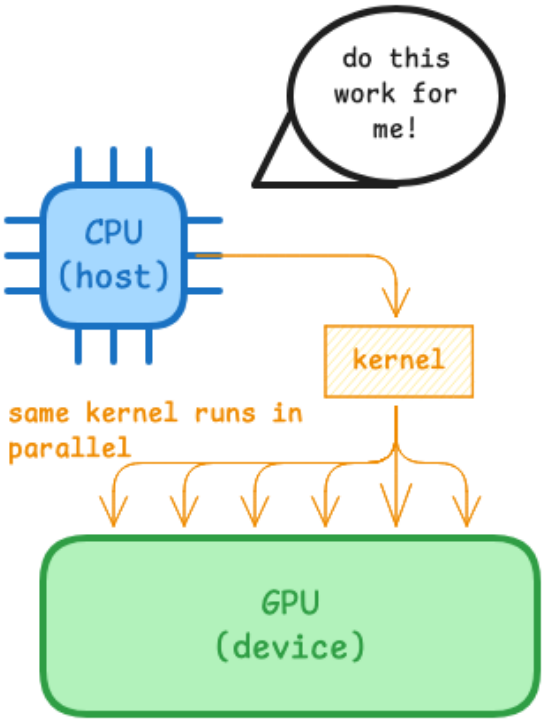
# CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



# CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.

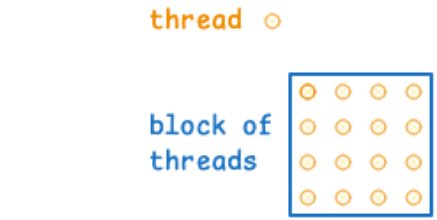



```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

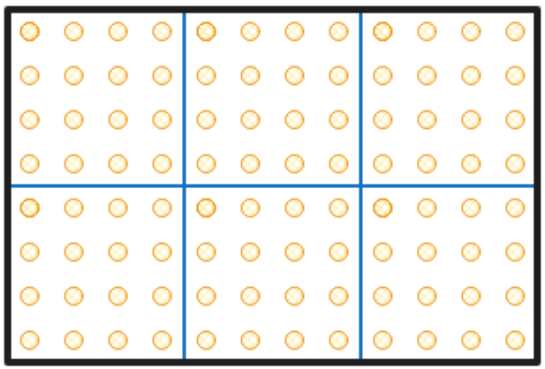
```
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

# CUDA Basics (Threads, Blocks, Grids)

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.




Grid of blocks



```cpp
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

```cpp
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

# Why Threads, Blocks, Grids?

**This seems complicated, why can't we just ask for $N$ threads?**

# Why Threads, Blocks, Grids?

**This seems complicated, why can't we just ask for $N$ threads?**

## Relation to hardware

This hierarchy is closely related to how the device schedules the execution. We will see later how to use this to optimize performance.

# Why Threads, Blocks, Grids?

**This seems complicated, why can't we just ask for $N$ threads?**

## Relation to hardware

This hierarchy is closely related to how the device schedules the execution.
We will see later how to use this to optimize performance.

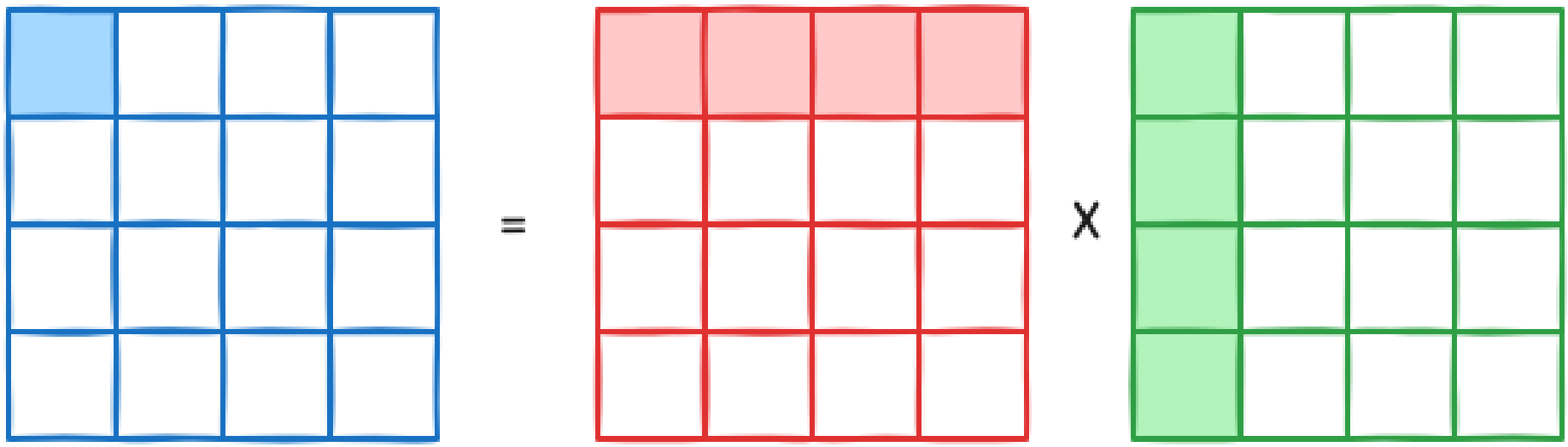
## Thread, Block, Grid hierarchy

Threads are grouped into blocks of constant size/shape.
Blocks are launched as a grid.
We choose the dimensions of the grid to control how many blocks are launched, and thus how many threads are launched.

# Let's design a SGEMM kernel!

## Recall

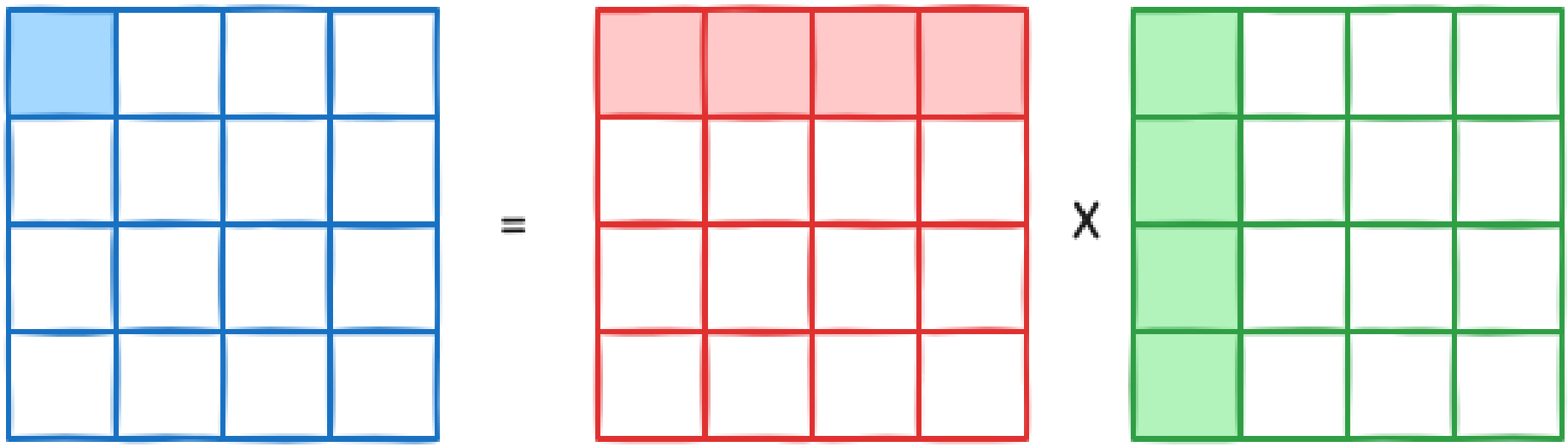
Recall that matrix multiplication is naturally parallel, and the output elements of $C$ can be computed independently in parallel.



# Let's design a SGEMM kernel!

## Recall

Recall that matrix multiplication is naturally parallel, and the output elements of $C$ can be computed independently in parallel.



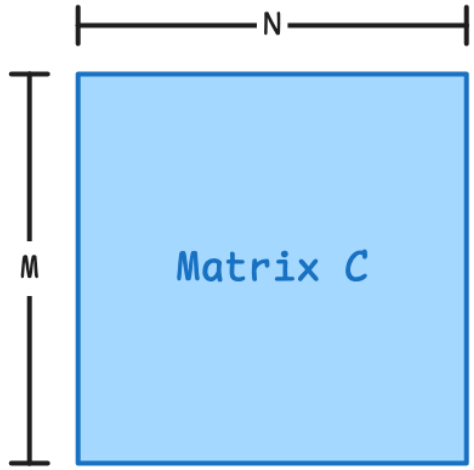
**We can naturally map one thread to each output element!**

# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

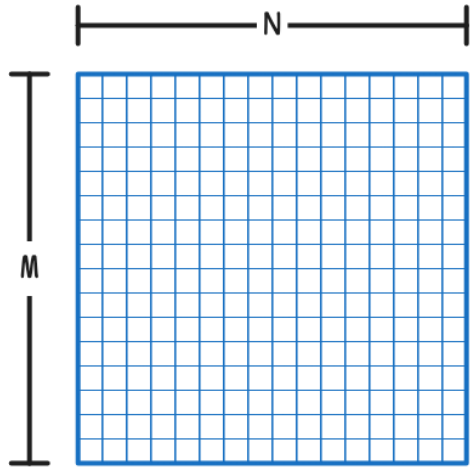


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

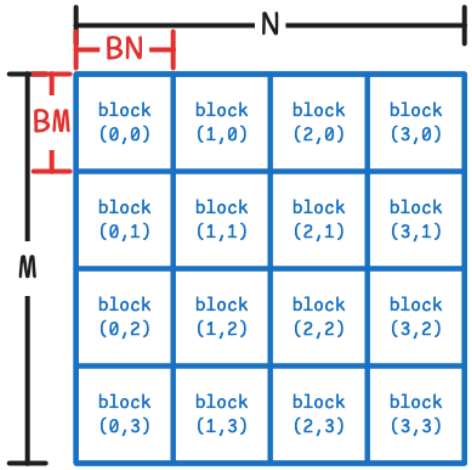


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

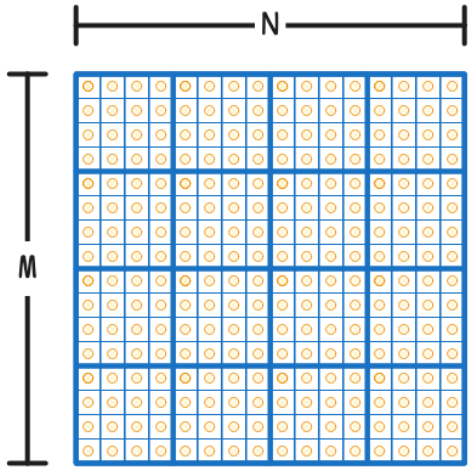


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

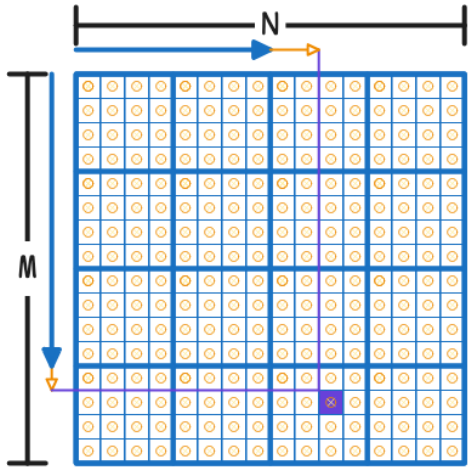


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

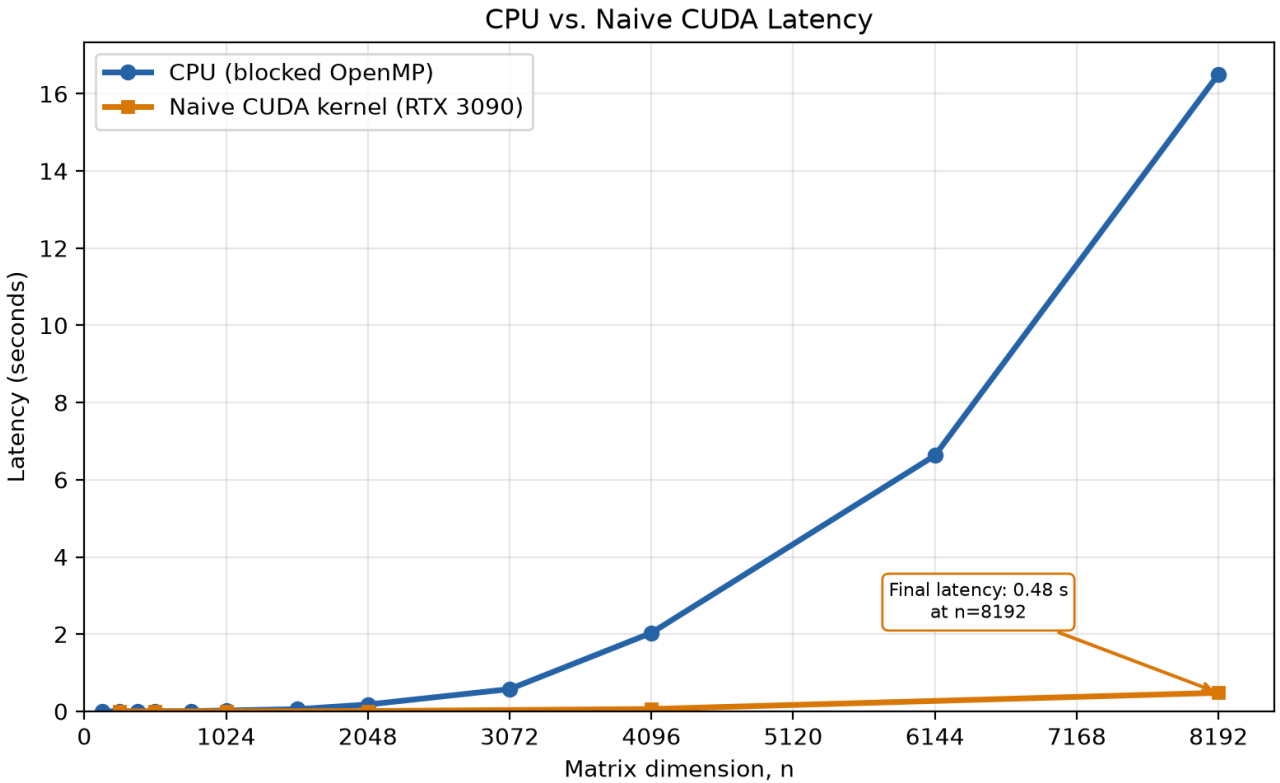


# Naive SGEMM visualization

- One thread computes one element of $C$.
- It walks one row of $A$ and one column of $B$ for the full dot product.

Play naive-kernel visualization

# Naive CUDA is already faster than the CPU

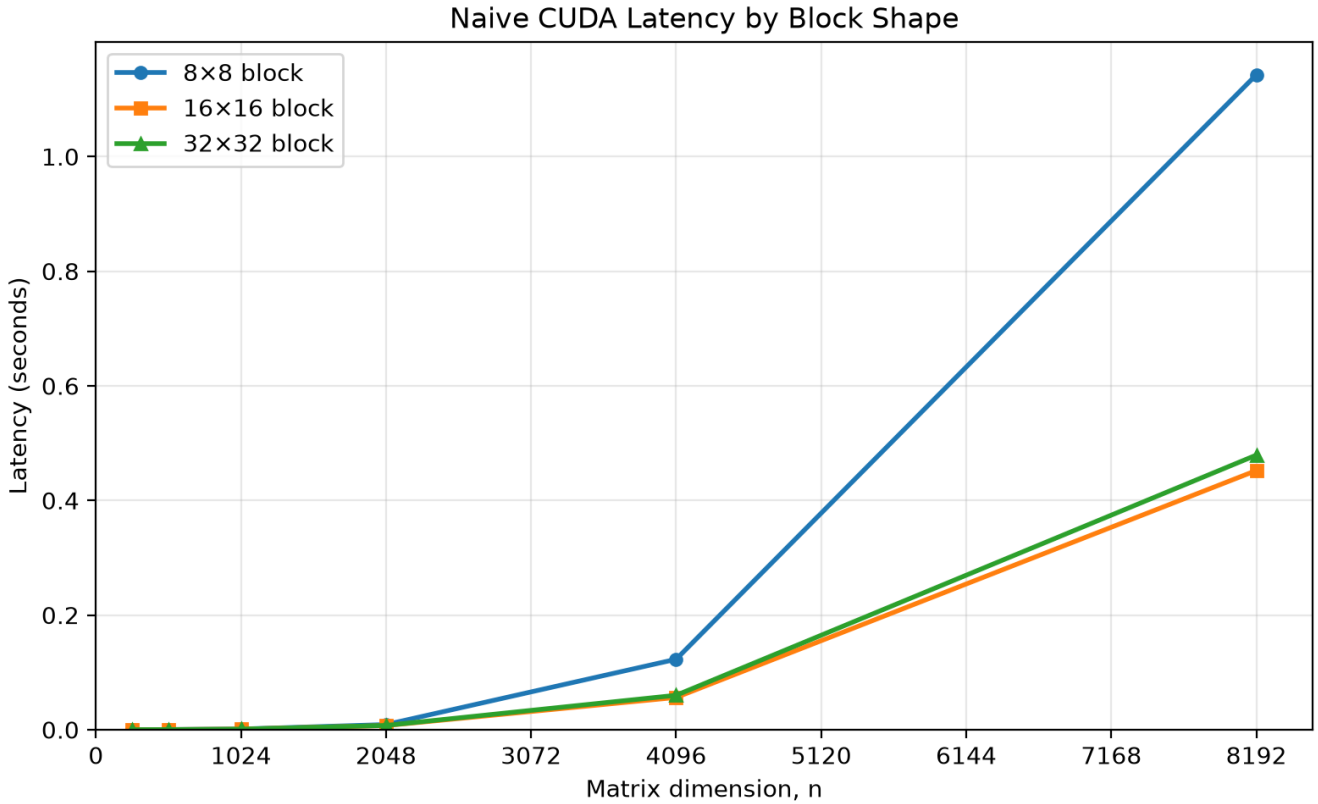


# Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

# Let's do some experimenting

**This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?**



# Blocks and SMs

## Interpretation

This happens because entire **blocks** are scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

# Blocks and SMs

## Interpretation

This happens because entire **blocks** are scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

| **Block shape** | **L1 load hit rate** |
|---|---|
| $8 \times 8$ | 83.51% |
| $16 \times 16$ | 87.13% |
| $32 \times 32$ | 94.98% |

RTX 3090, $M = N = K = 4096$

# Shared Memory (SMEM) Caching

## Programmable L1 Cache (SMEM)

Remember how our launch config required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so **threads in the same block have access to the SM's L1 cache/SMEM.**

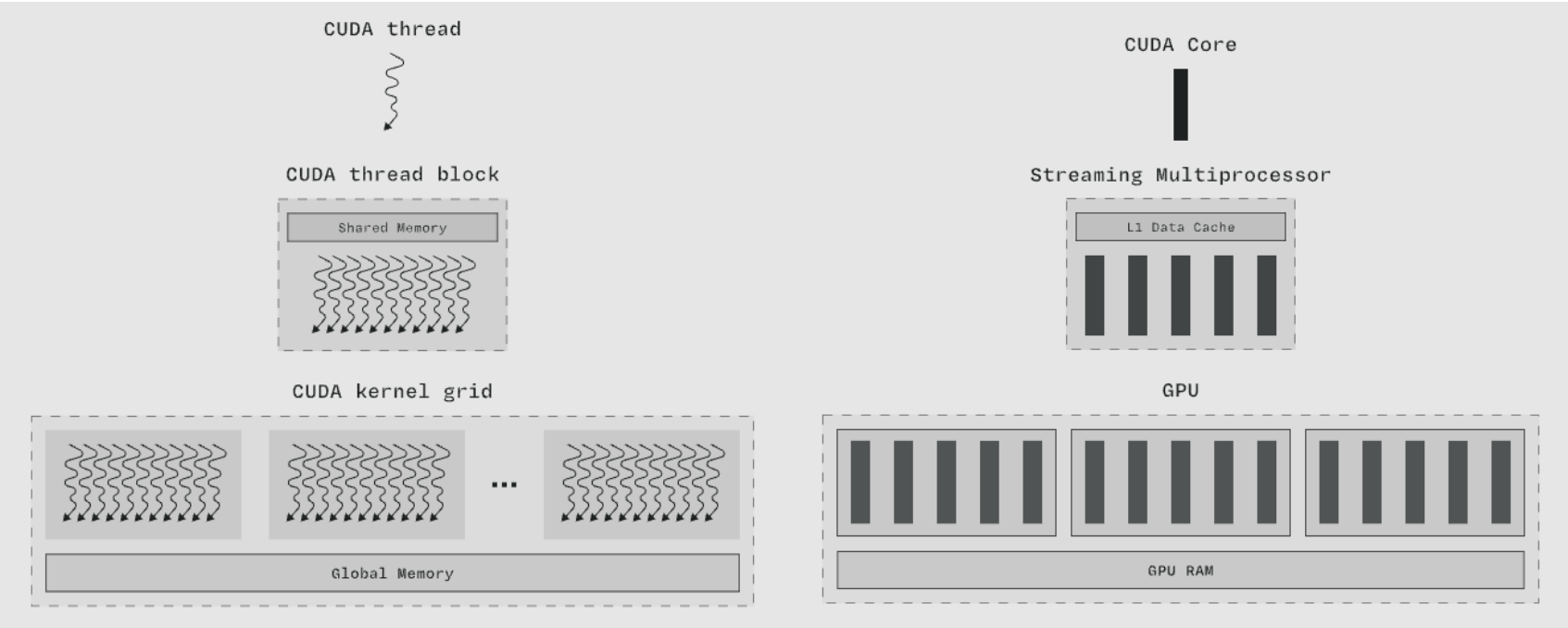



Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

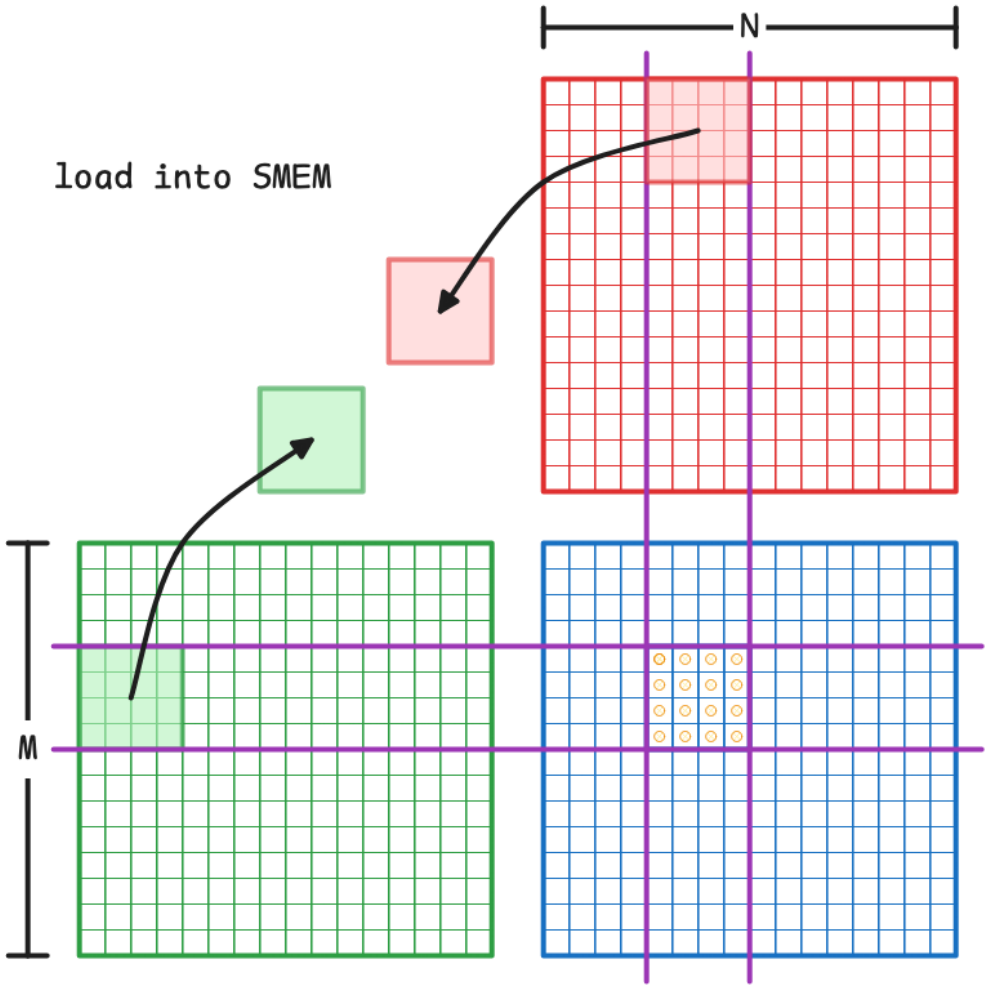
# Tiling

Explicit caching of data shared between threads.

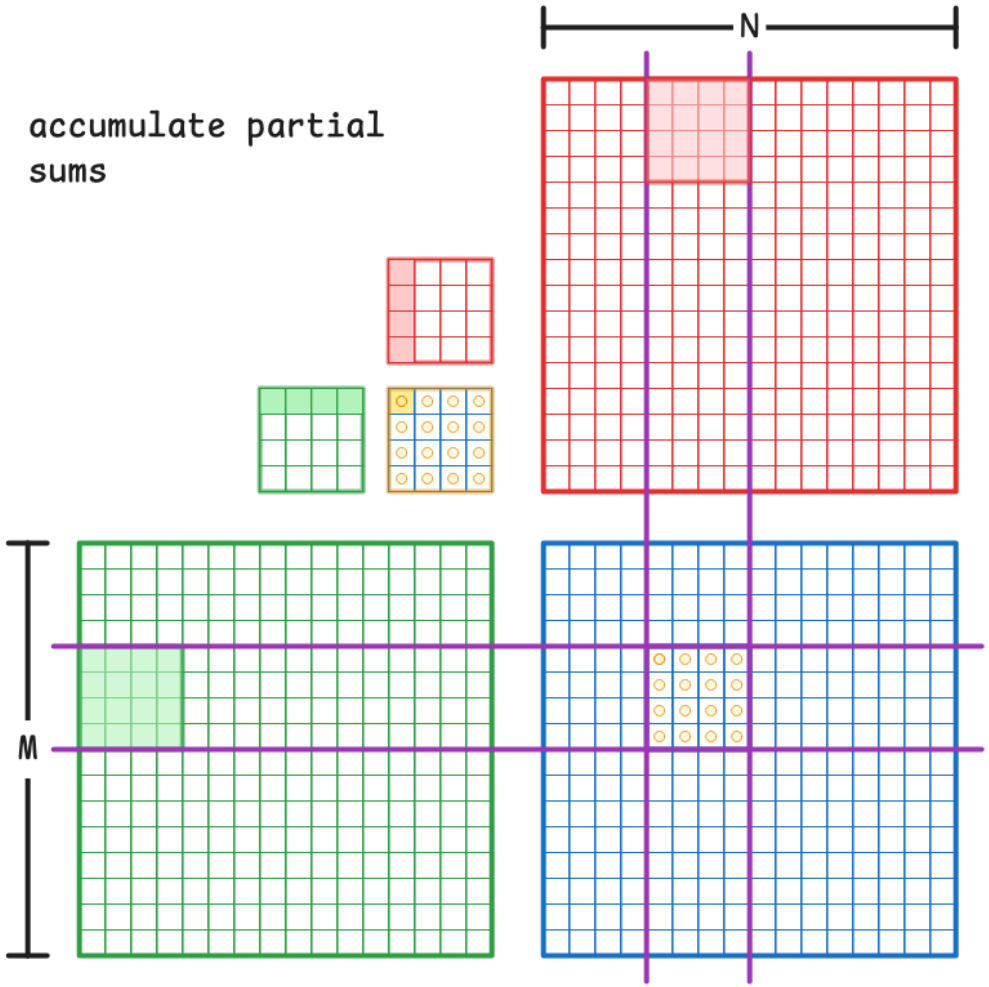
## Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

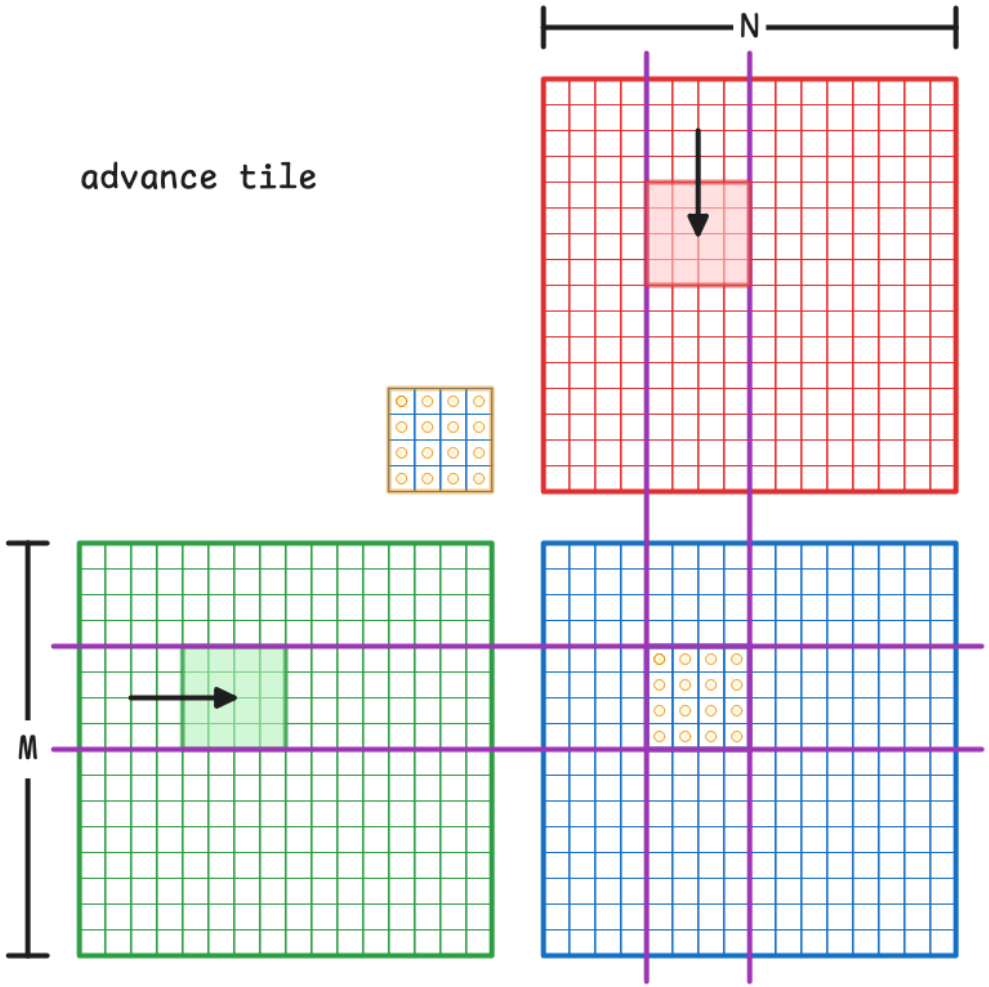
# Tiling steps



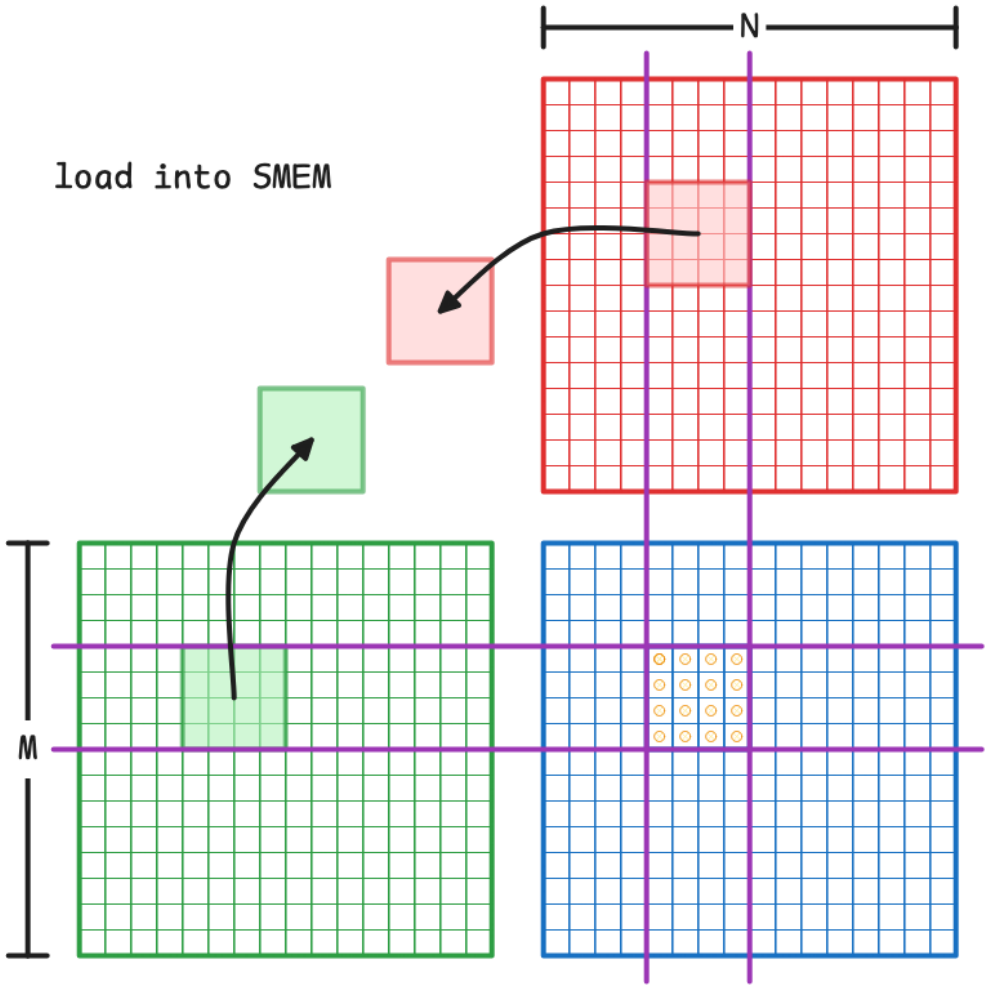
# Tiling steps



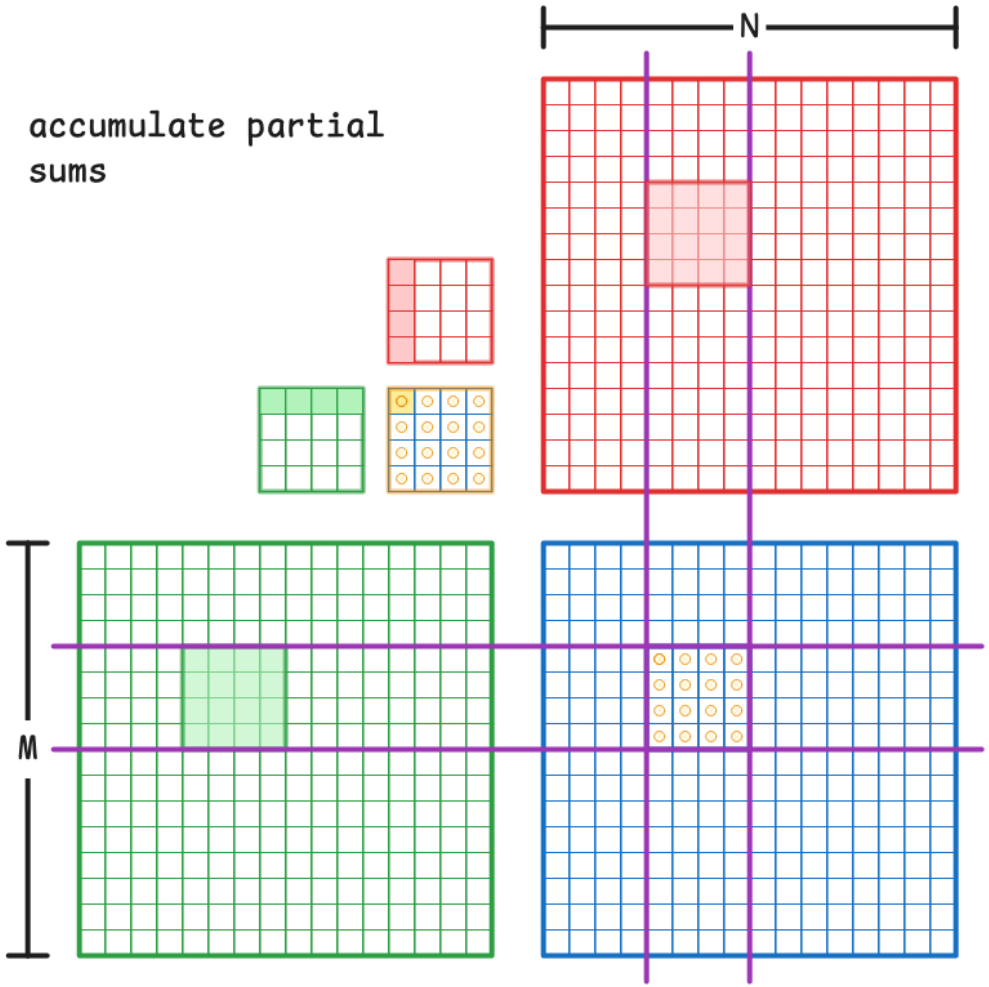
# Tiling steps



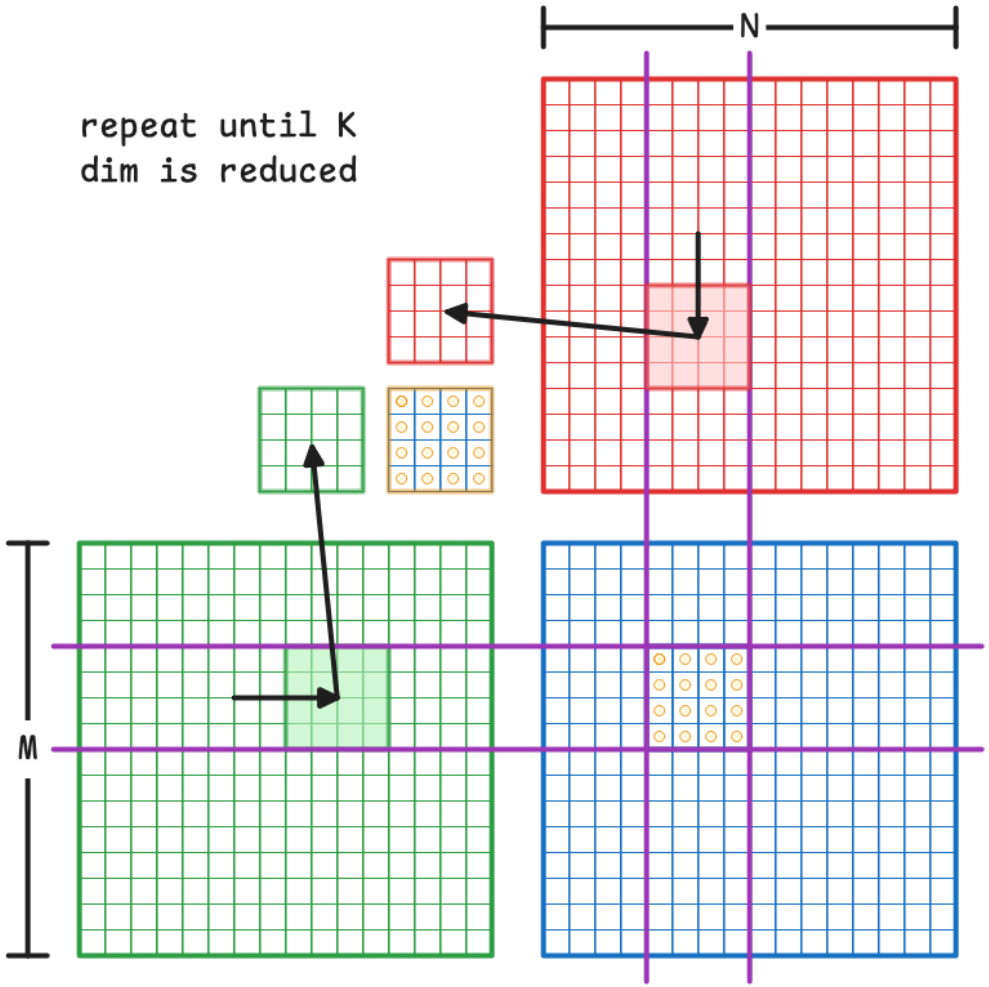
# Tiling steps



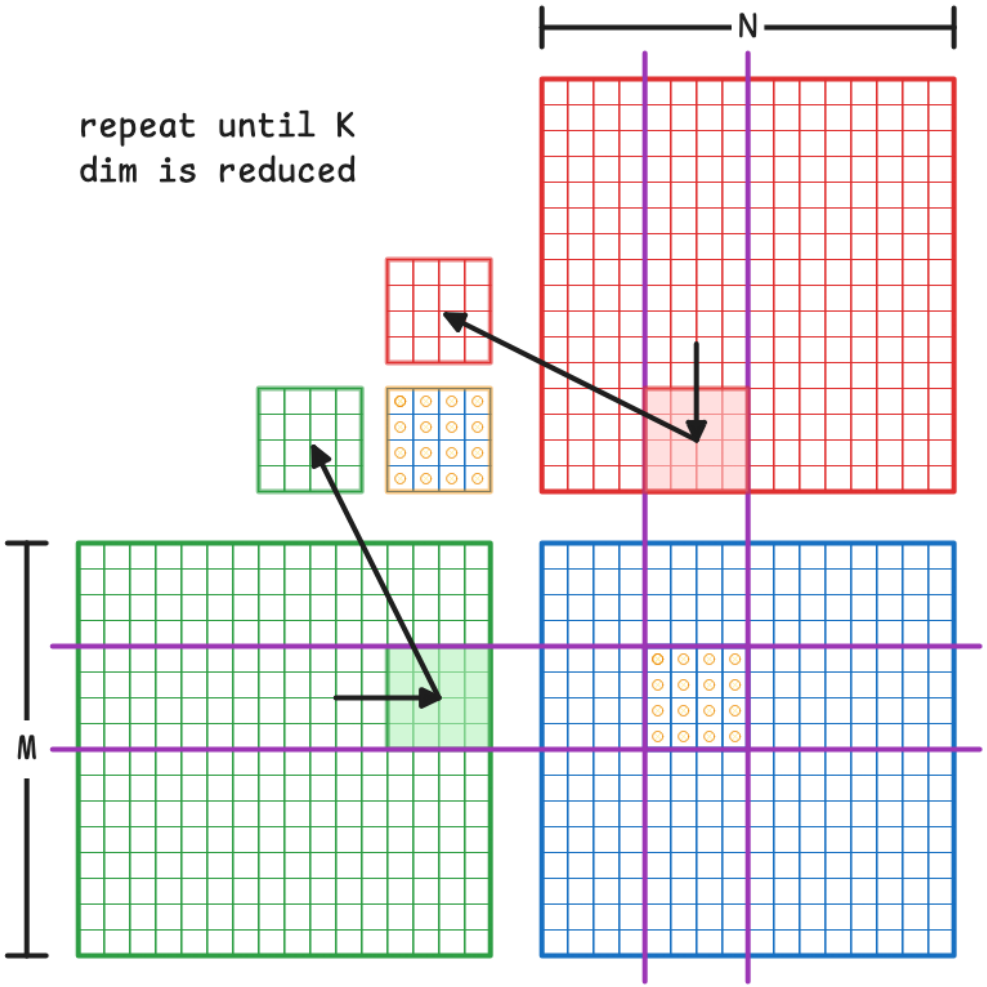
# Tiling steps



# Tiling steps



# Tiling steps



# Basic tiled SGEMM

```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {
  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }

  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```

**Outer loop:**
Loop over tiles.

**Cooperative loading to SMEM**

**Compute partial sum only with SMEM**

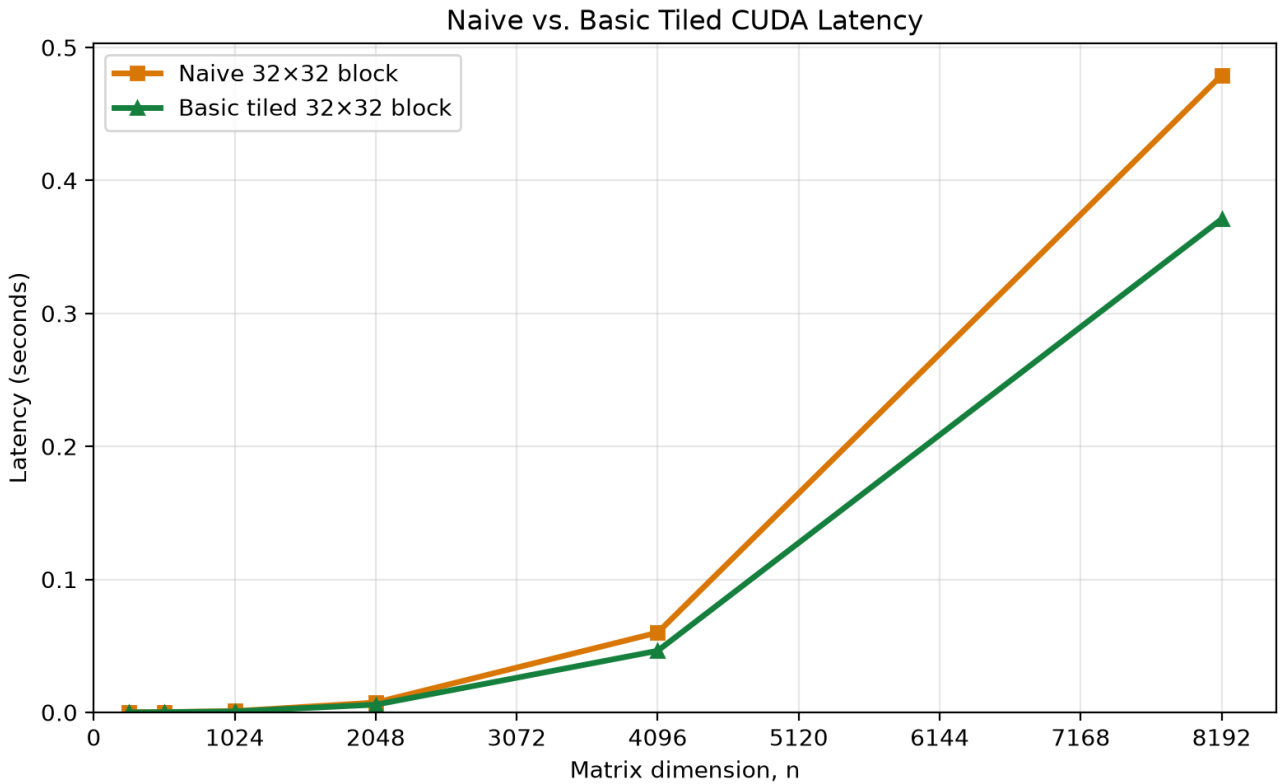
Source: `my_sgemm/src/sgemm.cu`.

# Basic tiled SGEMM visualization

- Watch each block load small regions of $A$ and $B$ together.
- The loaded tiles are reused to update an entire tile of $C$.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

# Tiling cuts latency by another order of magnitude




At $n, m, k = 8192$, tiled 32×32 latency is 371 ms, a 22.6% improvement over the naive.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

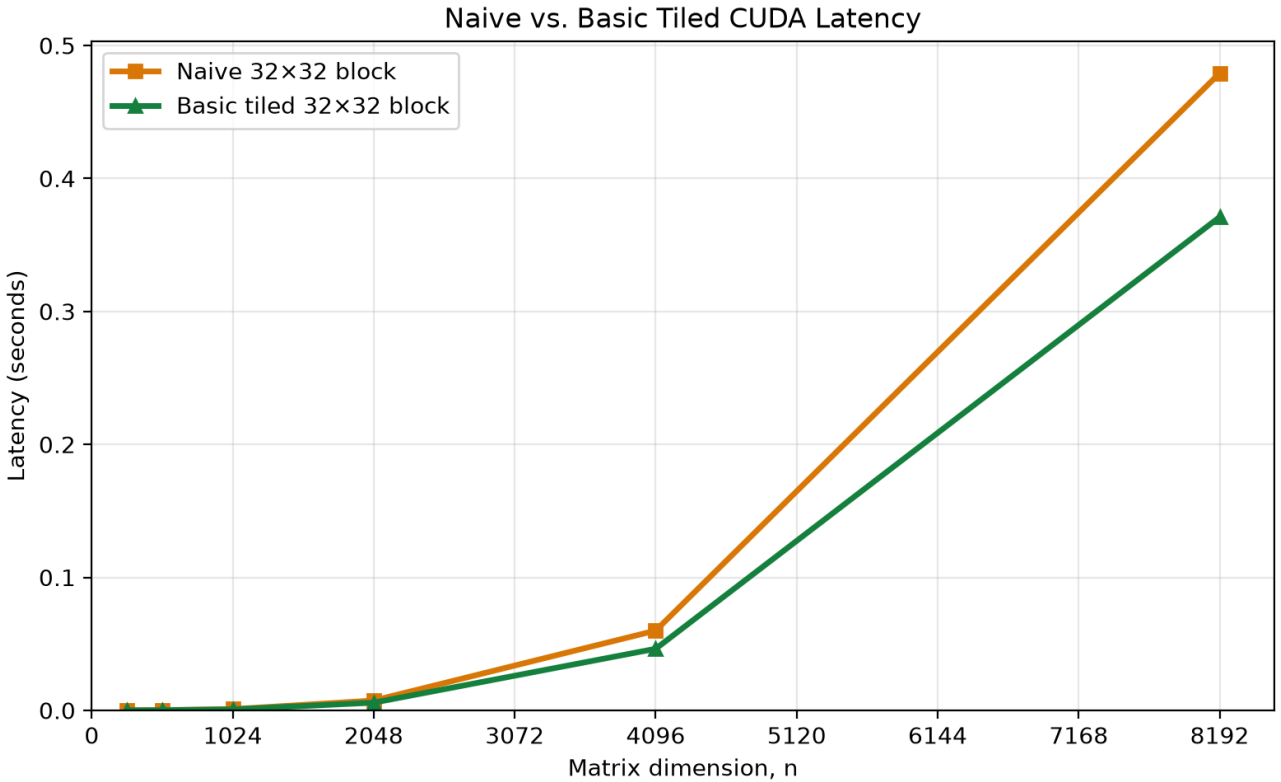
At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM. **40% of our latency is moving data.**

# Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

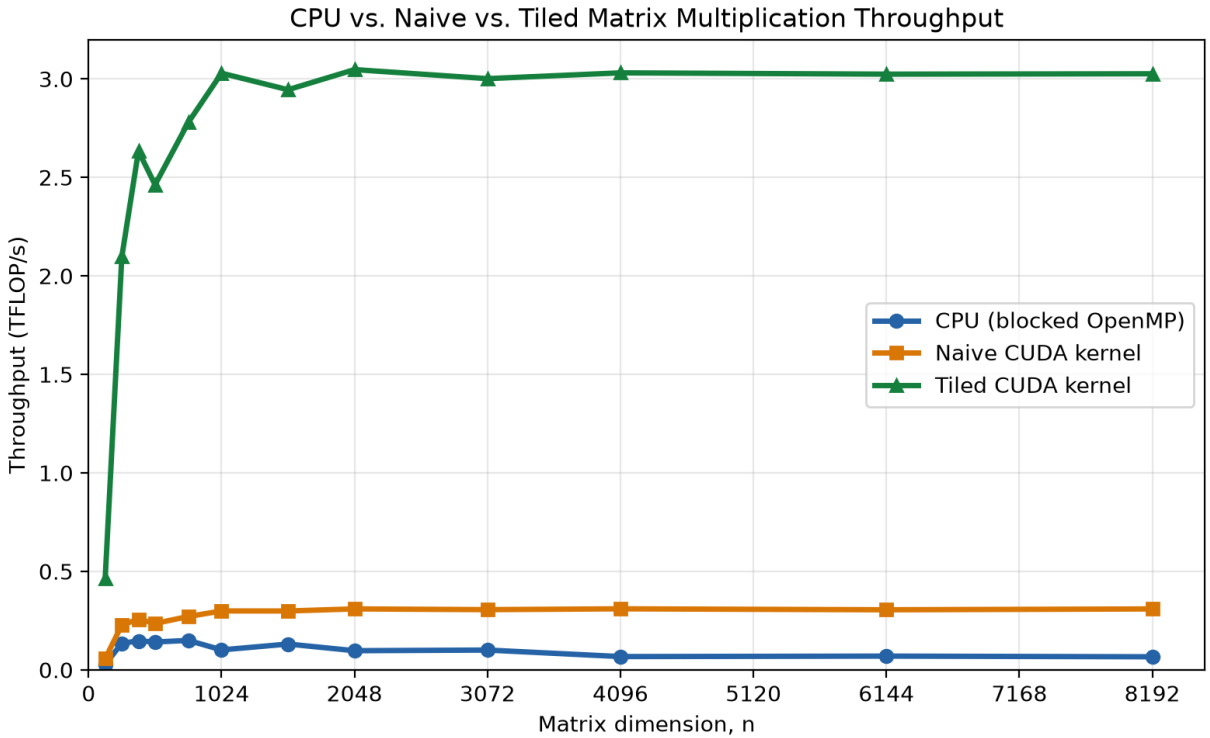
# Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.



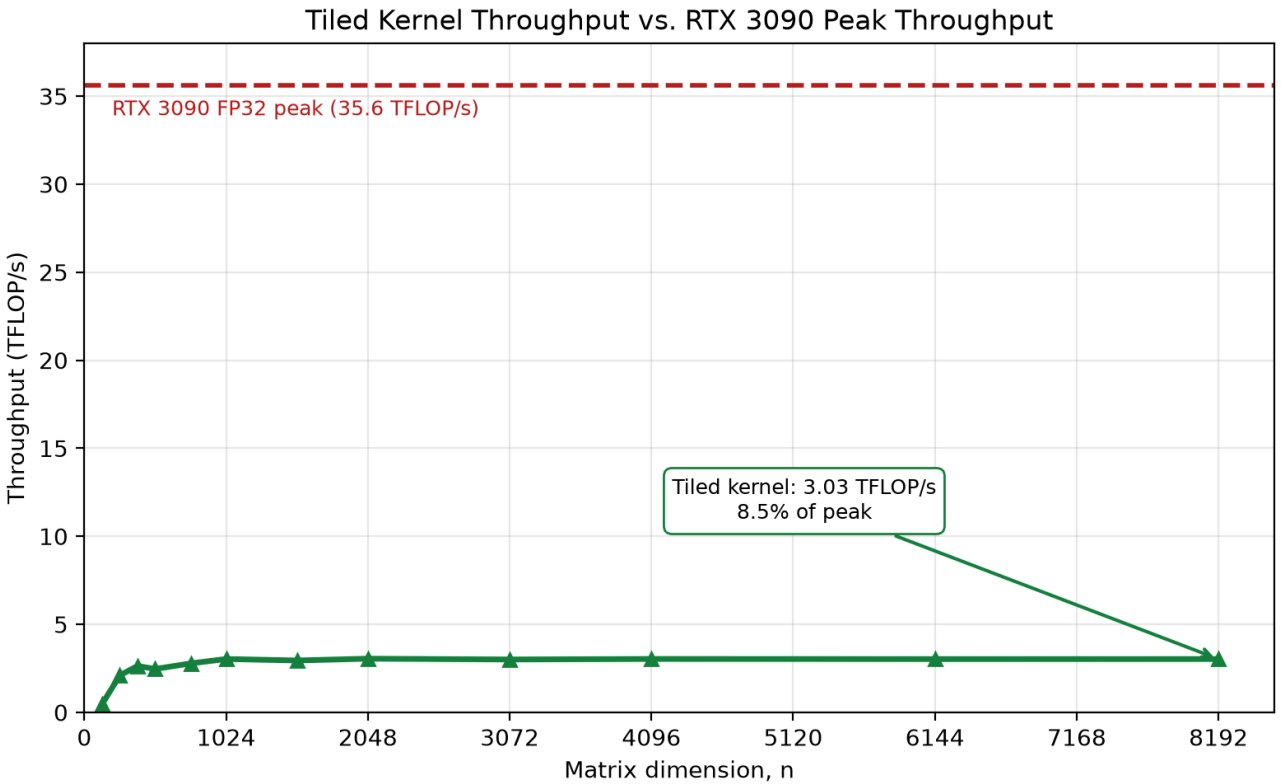
# Why latency alone isn't enough

- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.



# Why latency alone isn't enough

- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.



# Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.
- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.
- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.

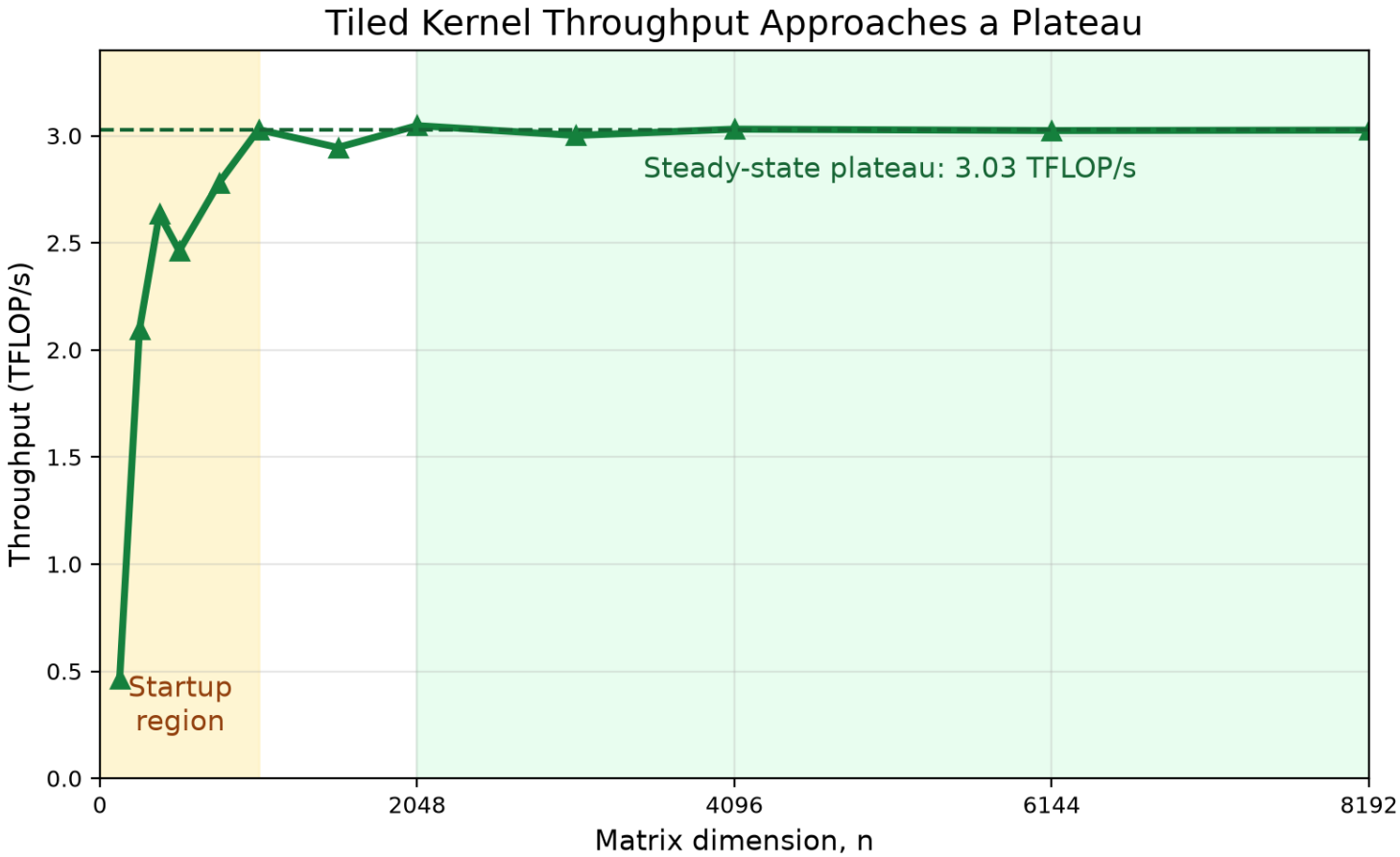
## Why throughput

Throughput tells us

1. A general performance metric, no matter the problem size
2. How well we are utilizing hardware

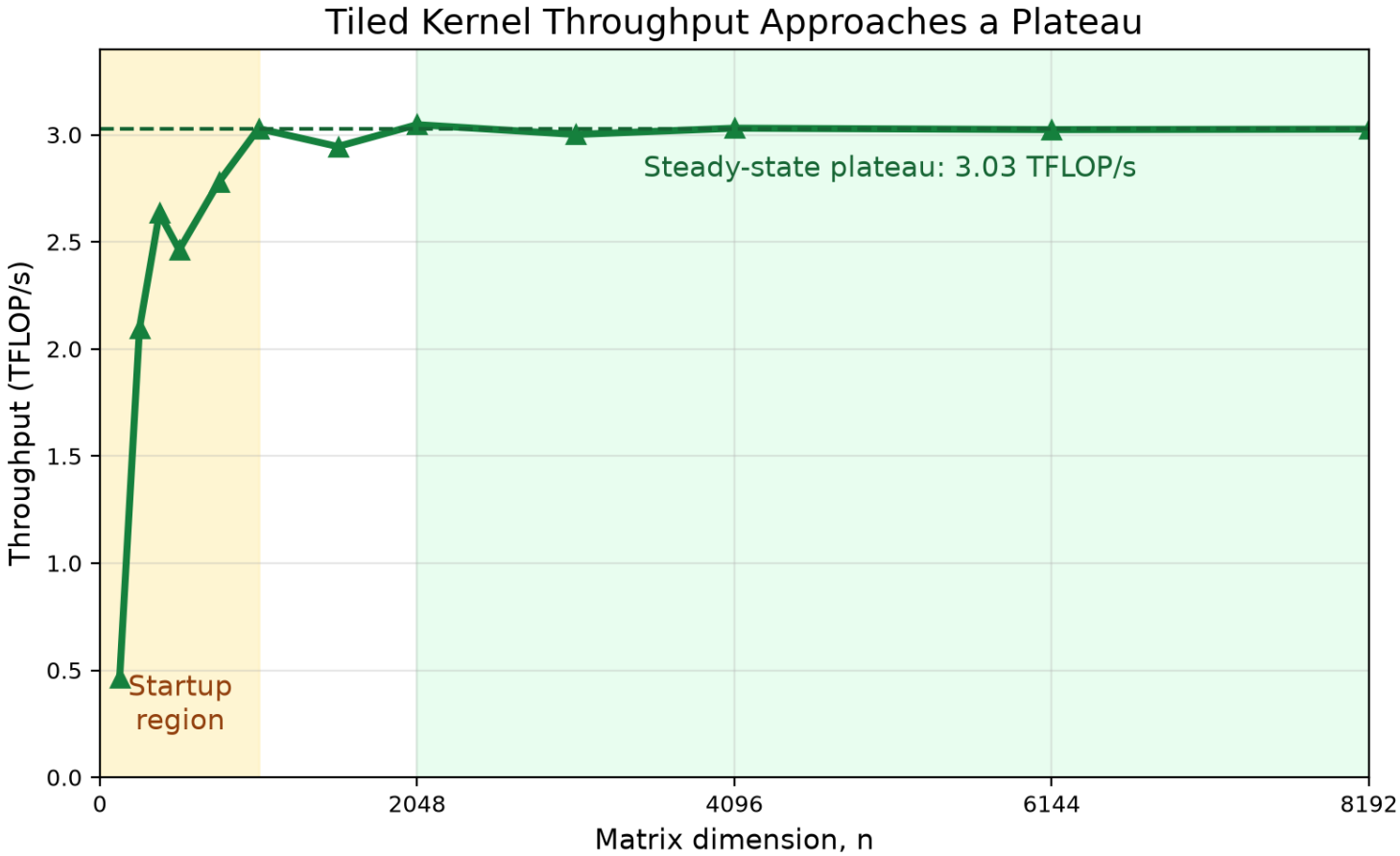
# Why throughput still depends on problem size

Normalizing by useful work makes comparisons fairer, not size-independent.



# Why throughput still depends on problem size

Normalizing by useful work makes comparisons fairer, not size-independent.




## Measurement rule

Measure in the plateau region, and report the problem size with the result.

# Throughput as a performance measure

## Throughput

**Throughput** is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

## Example

Peak throughput of our RTX 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs. Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\text{TFLOP/s}.$$

Roughly 8.5% of peak throughput.

# Arithmetic Intensity

## Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

## Example

For $n, k, m = 8192$, our tiled kernel moved a total of 137.7 GB of data from DRAM. With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \times 10^{12} \text{ FLOP}}{137.7 \times 10^{9} \text{ bytes}} \approx \boxed{8.0 \text{ FLOP/byte}}.$$

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our **memory roof** is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} \approx 7.49 \text{ TFLOP/s}.$$

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our **memory roof** is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} \approx 7.49 \text{ TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 TFLOP/s.

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our **memory roof** is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} \approx 7.49 \text{ TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 TFLOP/s. **No matter how we optimize this kernel, without increasing AI, we will never achieve greater than 7.49 TFLOP/s**.

# Hypothetical Kernel

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s.

# Hypothetical Kernel

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our upper bound throughput is now

$$935.8 \text{ GB/s} \times 16.0 \text{ FLOP/byte} \approx 15.0 \text{ TFLOP/s}.$$

# Hypothetical Kernel

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our upper bound throughput is now

$$935.8 \text{ GB/s} \times 16.0 \text{ FLOP/byte} \approx 15.0 \text{ TFLOP/s}.$$

Since $15.0 \text{ TFLOP/s} < 35.6 \text{ TFLOP/s}$, this kernel will still be memory bound.

# Hypothetical Kernels

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

| AI (FLOP/byte) | Upper bound throughput (TFLOP/s) |
|---|---|
| 8 | 7.49 |
| 16 | 14.97 |
| 32 | 29.95 |
| 36 | 33.69 |
| 38 | 35.56 |
| 40 | 37.43 |

# Hypothetical Kernels

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

| AI (FLOP/byte) | Upper bound throughput (TFLOP/s) |
|---|---|
| 8 | 7.49 |
| 16 | 14.97 |
| 32 | 29.95 |
| 36 | 33.69 |
| 38 | 35.56 |
| 40 | 37.43 |

Wait! Our peak throughput is 35.6 TFLOPs! At AI $\geq$ 38, we are **compute bound**. Our data can finally keep up with the CUDA Cores!

# Ridge Point

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

AI = 38 FLOP/byte is known as the **ridge point** *RP*, and can be found by solving:

$$\text{Data roof} = \text{Peak throughput}$$

$$\text{DRAM Bandwidth} \times \text{RP} = \text{Peak throughput}$$

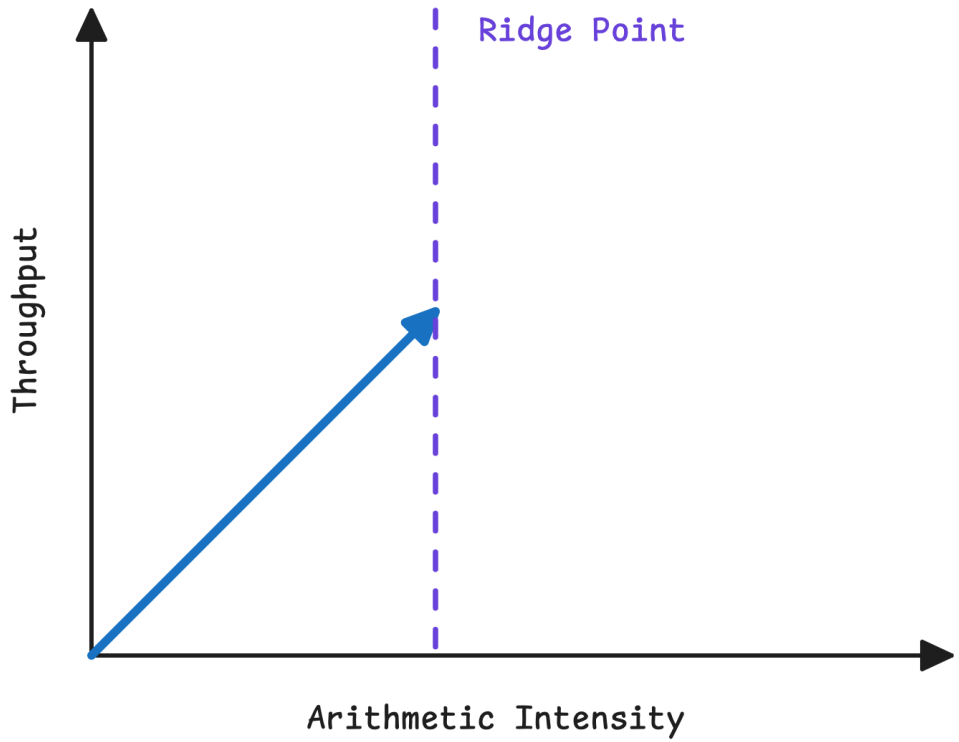
$$\text{RP} = \frac{\text{Peak throughput}}{\text{DRAM Bandwidth}}$$

$$\text{RP} = \frac{35.6 \text{ TFLOP/s}}{935.8 \text{ GB/s}} \approx 38.0 \text{ FLOP/s}$$

# Roofline Model

## Roofline Model

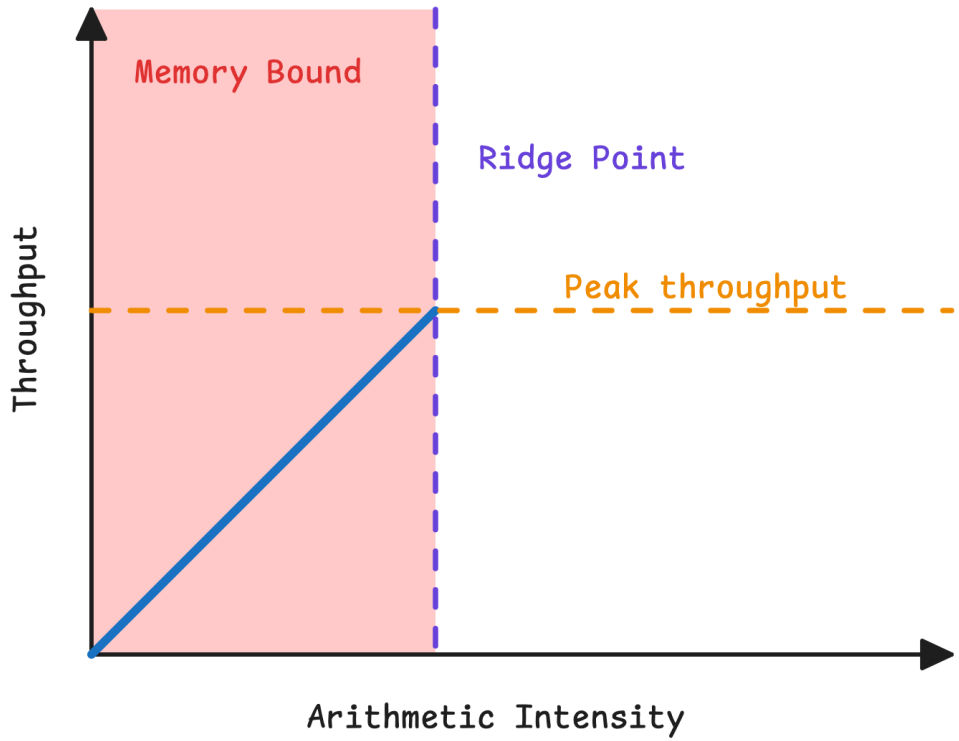
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI.



# Roofline Model

## Roofline Model

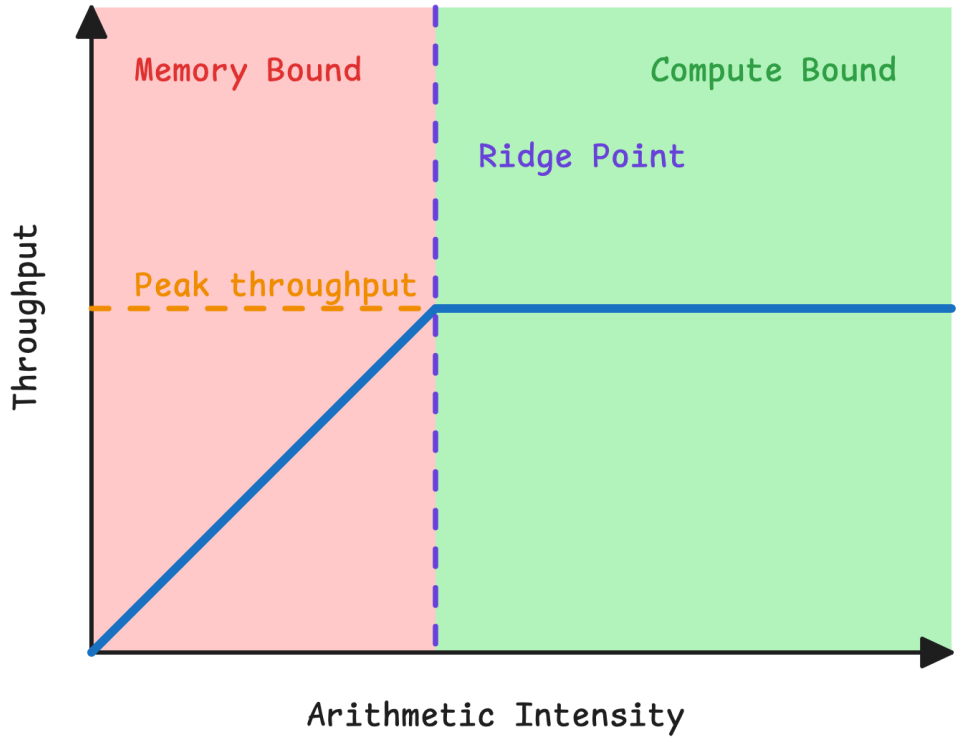
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput.



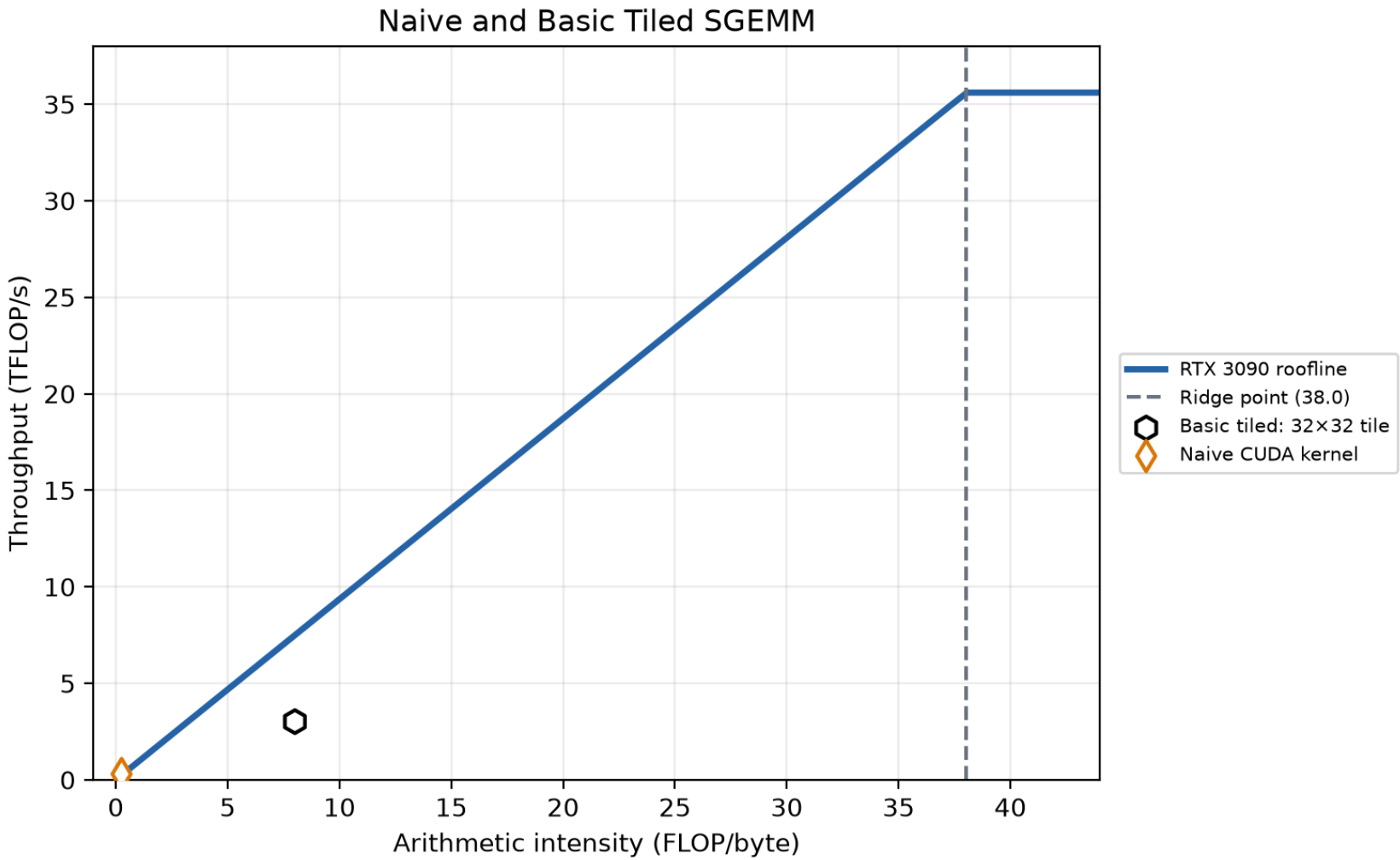
# Roofline Model

## Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. Past that, we are **compute bound**, increasing AI will not raise the upper bound of throughput.



# Roofline Model of our RTX 3090

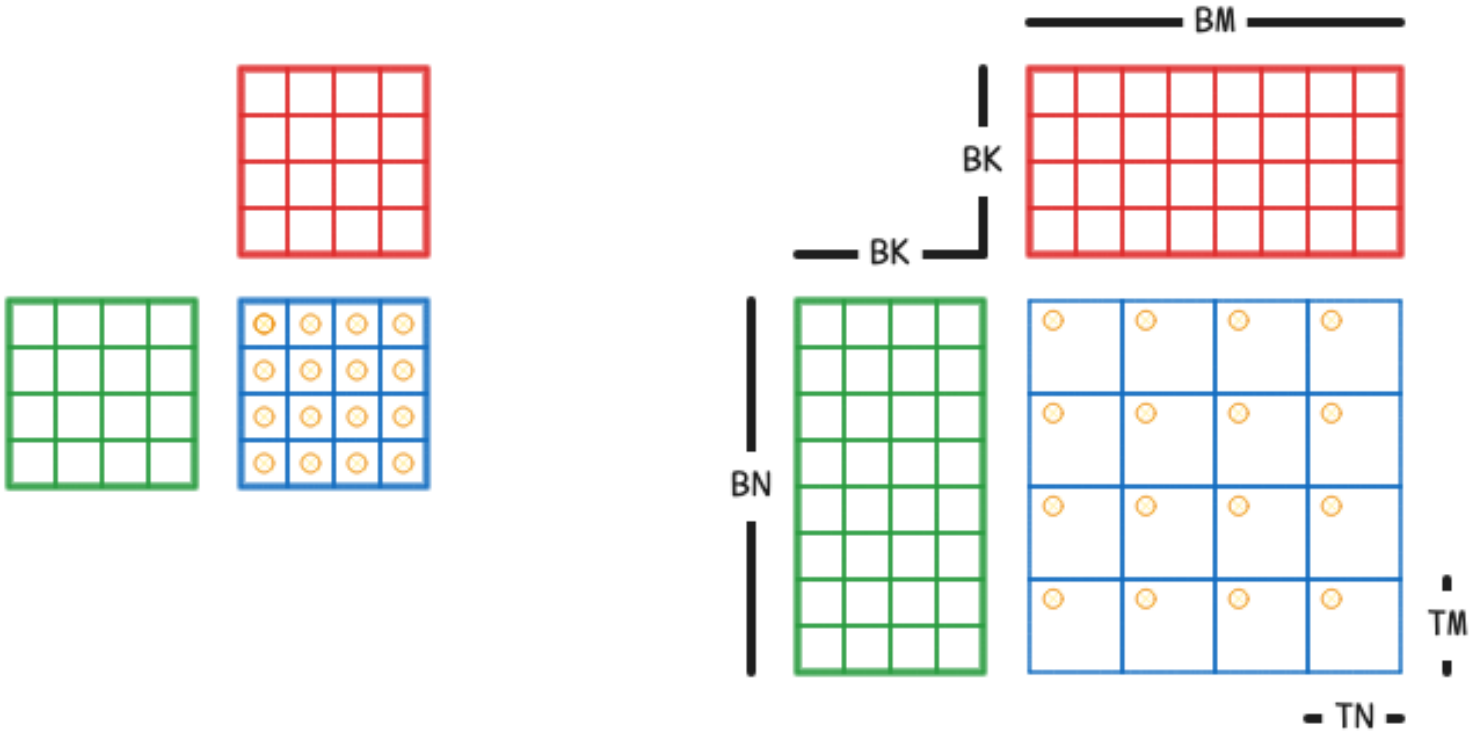


# Section Overview

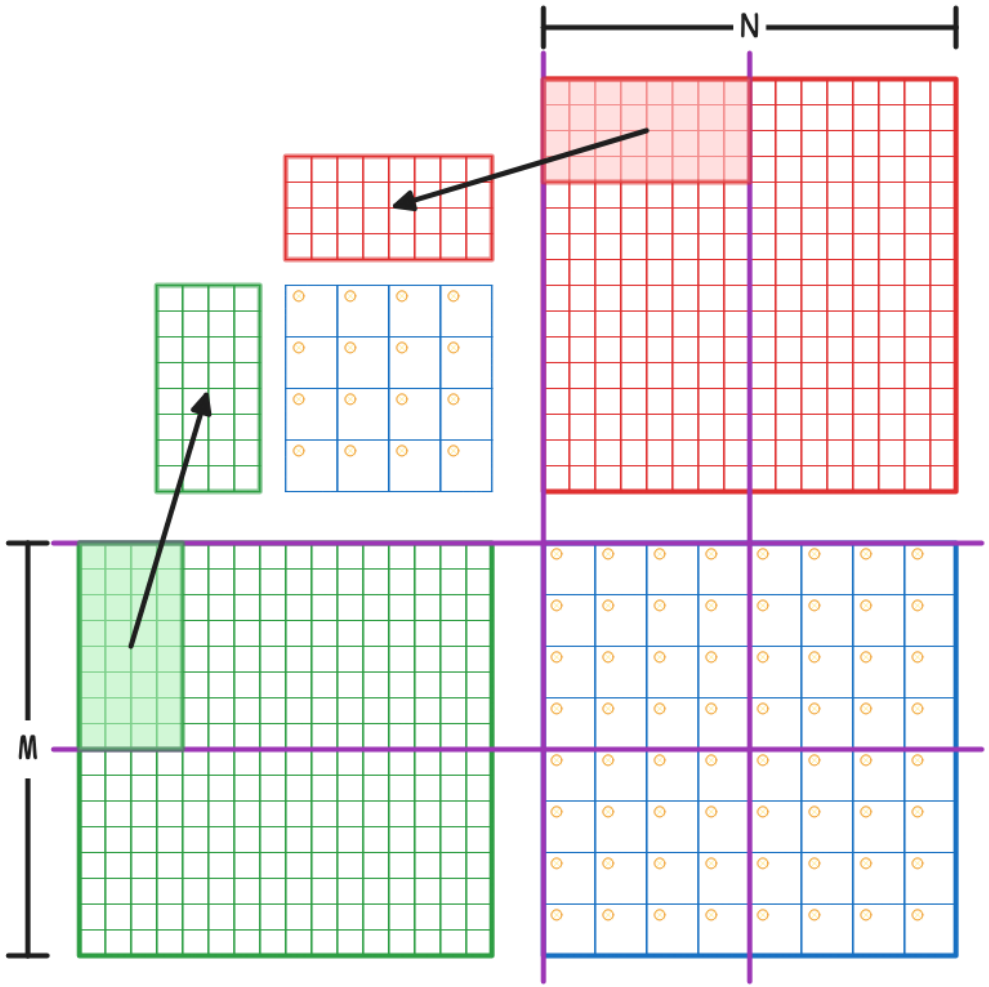
1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

# Thread coarsening: Increasing AI

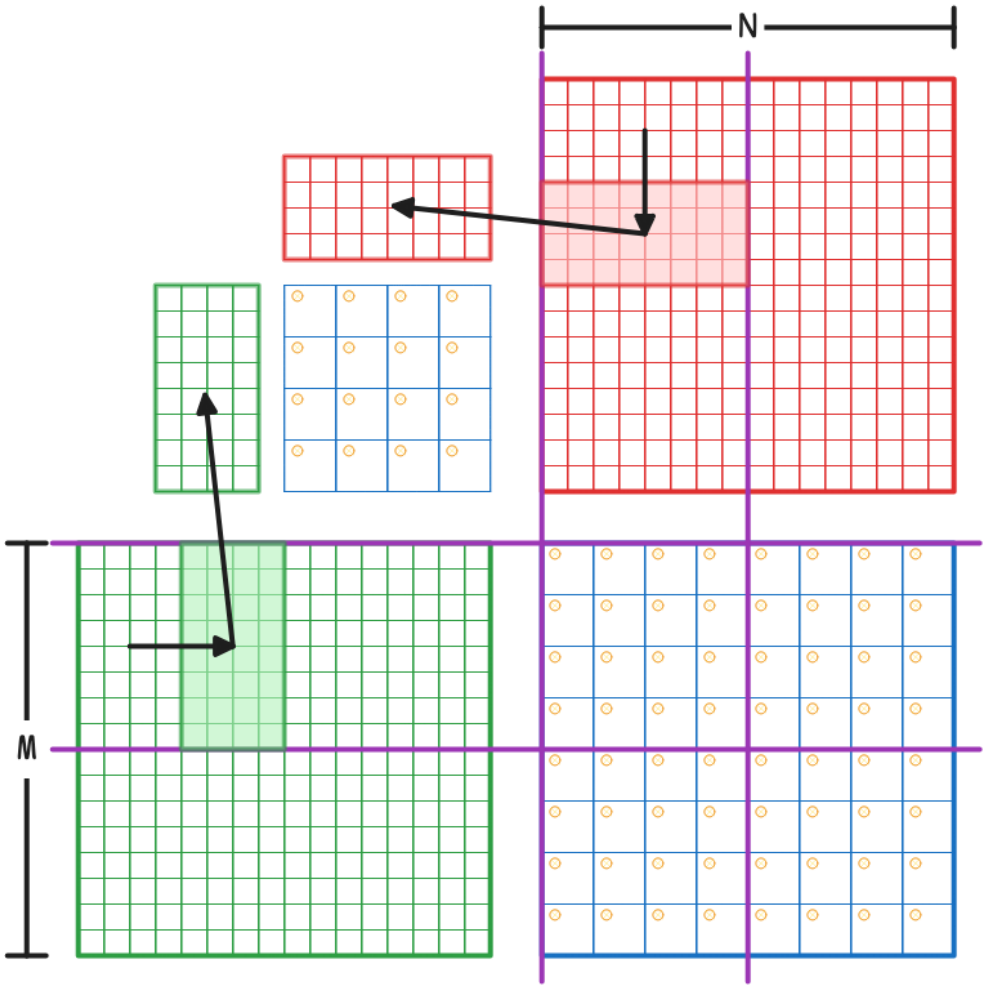
- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded $A$ value can update several columns; one loaded $B$ value can update several rows.



# Coarsened SGEMM visualization



# Coarsened SGEMM visualization



# Coarsening factors are a tradeoff

## Two levels of tiling

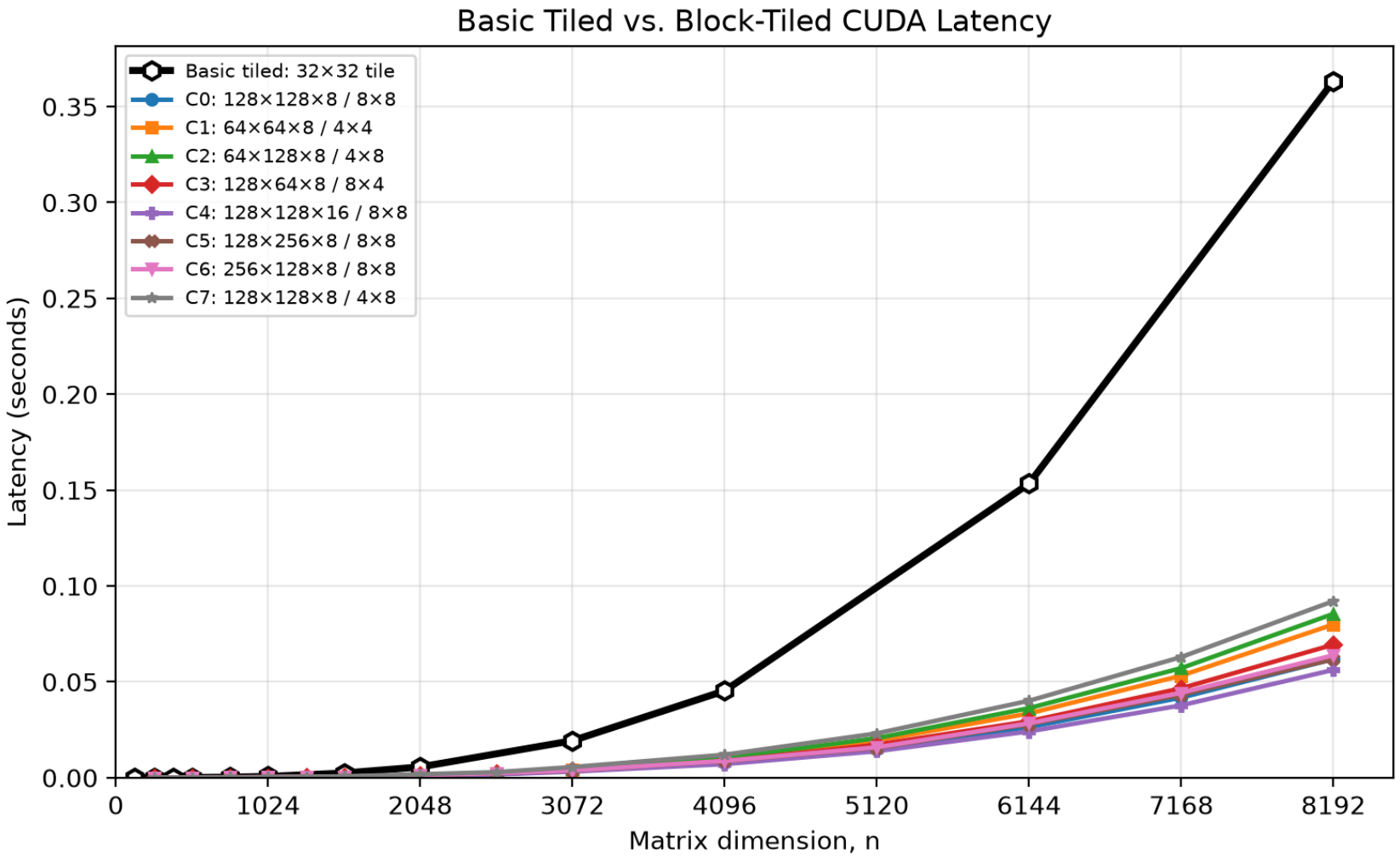
- A block computes a $BM \times BN$ output tile while advancing through $K$ in steps of $BK$.
- Each thread computes a $TM \times TN$ micro-tile in registers.
- Thread count: $\frac{BM}{TM}\frac{BN}{TN}$.

## Why not keep increasing them?

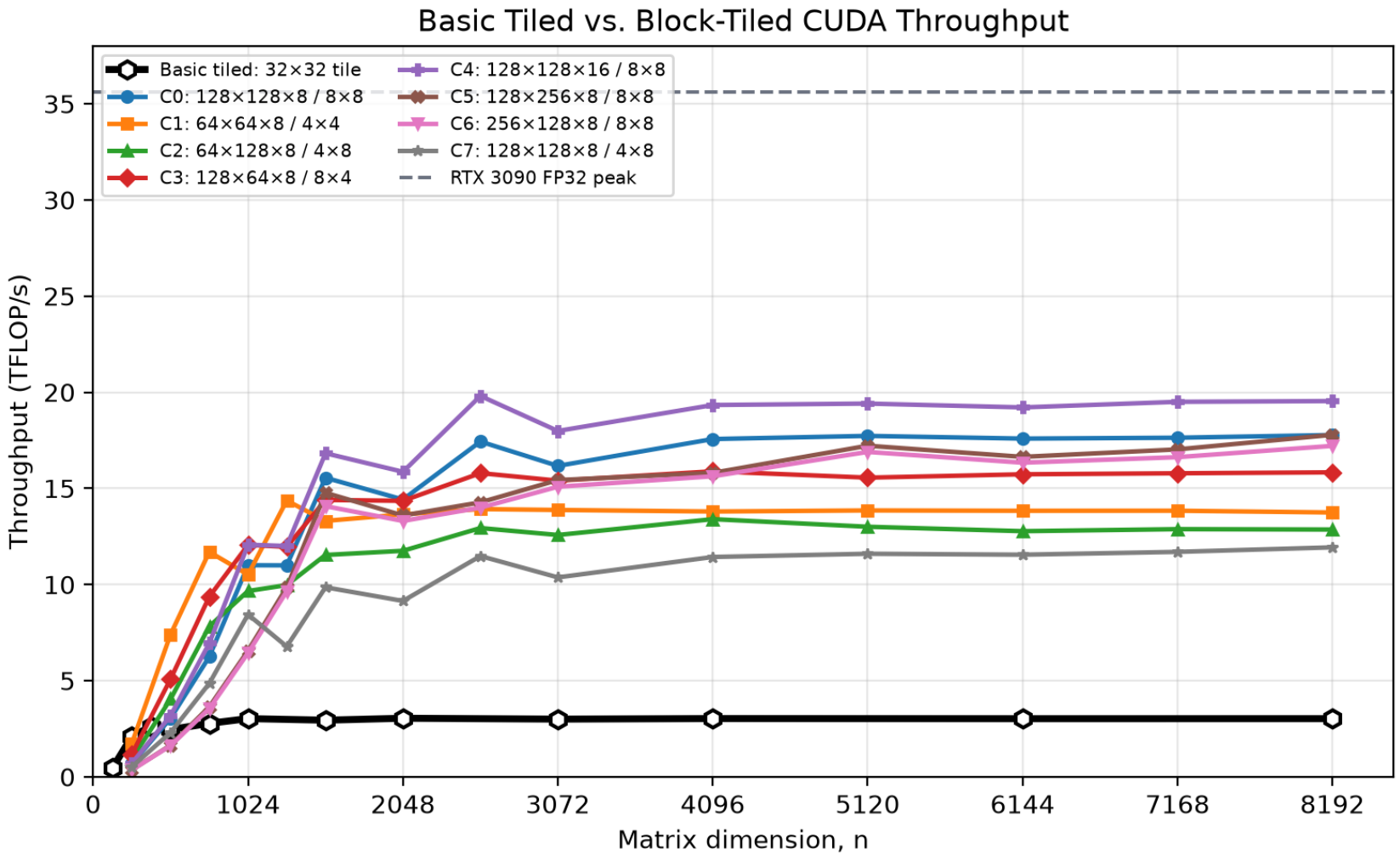
Larger tiles improve reuse and arithmetic intensity, but consume more registers and shared memory. Eventually occupancy falls and extra parallelism or reuse no longer helps.

| Config. | $BM \times BN \times BK$ | $TM \times TN$ | AI | Throughput |
|---|---|---|---|---|
| C1 | $64 \times 64 \times 8$ | $4 \times 4$ | 15.9 | 13.75 TFLOP/s |
| C4 | $128 \times 128 \times 16$ | $8 \times 8$ | 31.5 | **19.54 TFLOP/s** |
| C5 | $128 \times 256 \times 8$ | $8 \times 8$ | 41.8 | 17.79 TFLOP/s |

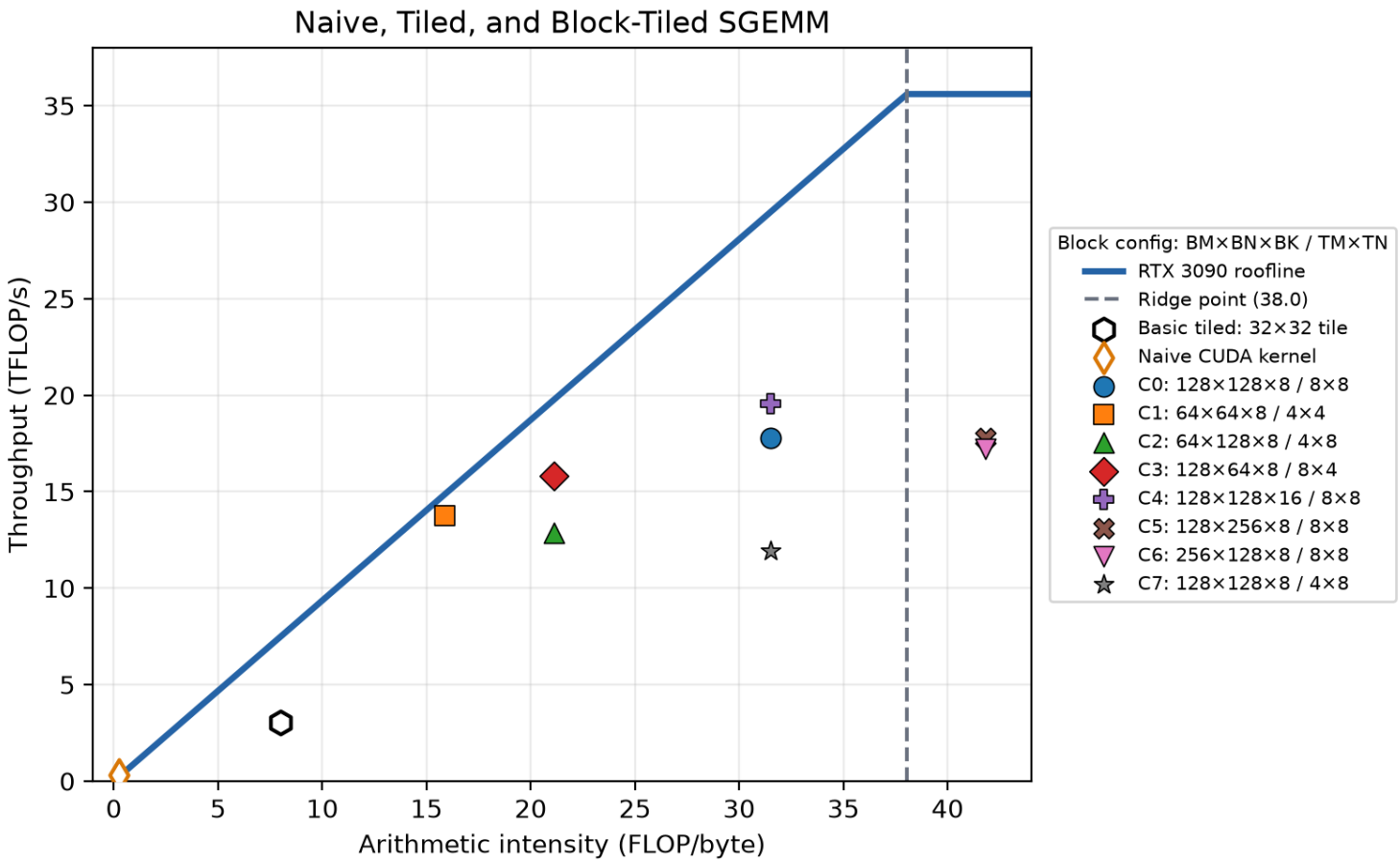
# Basic tiled vs. block-tiled latency



# Block tiling raises useful throughput



# Tiling configurations on the roofline




Reminder that the Roofline Model only tells us the upper bound!

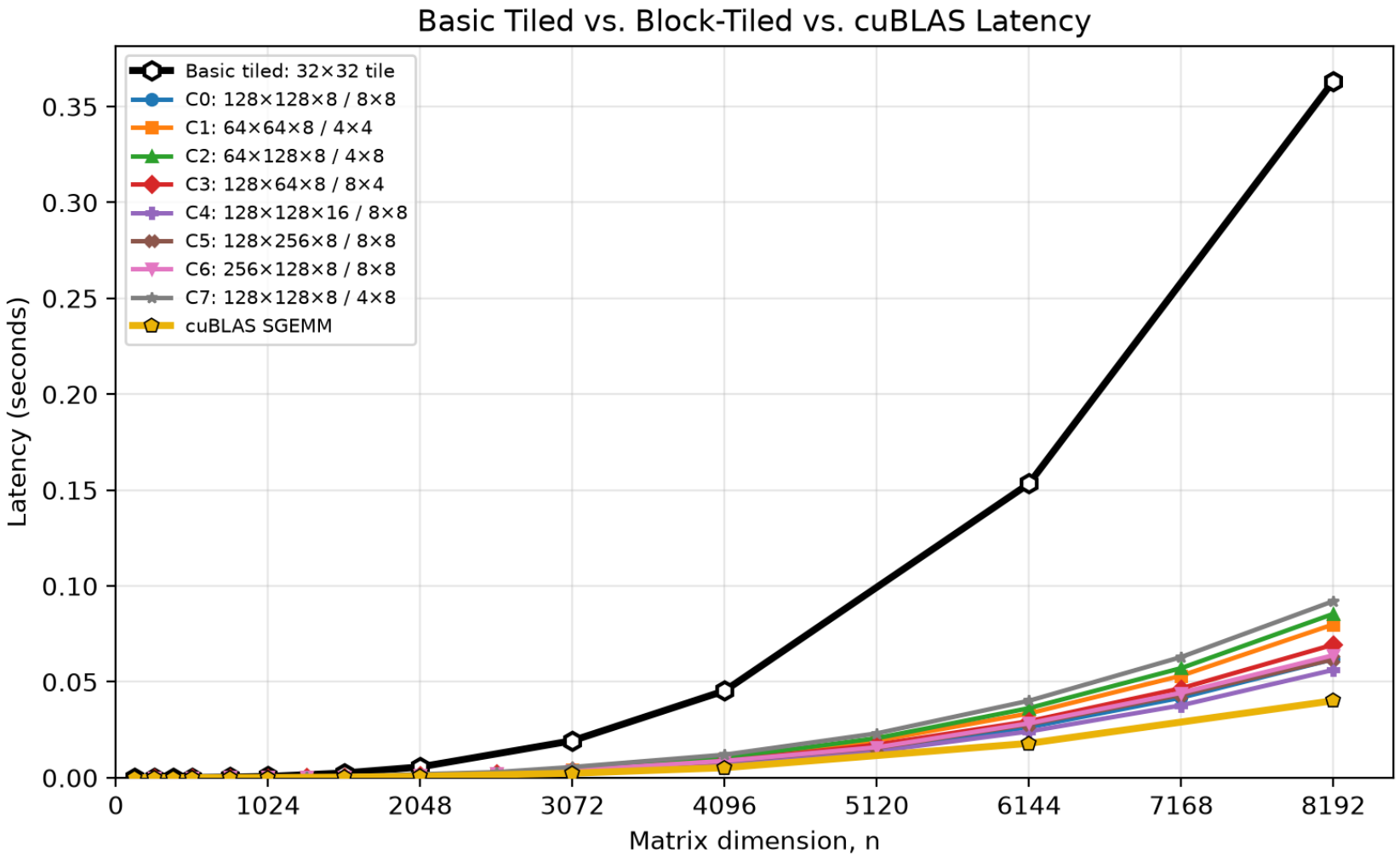
# We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 27.38 TFLOP/s, while our fastest configuration (C4) reaches 19.54 TFLOP/s, or 71.4% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.2 ms versus 56.3 ms for C4, about 1.40× faster.

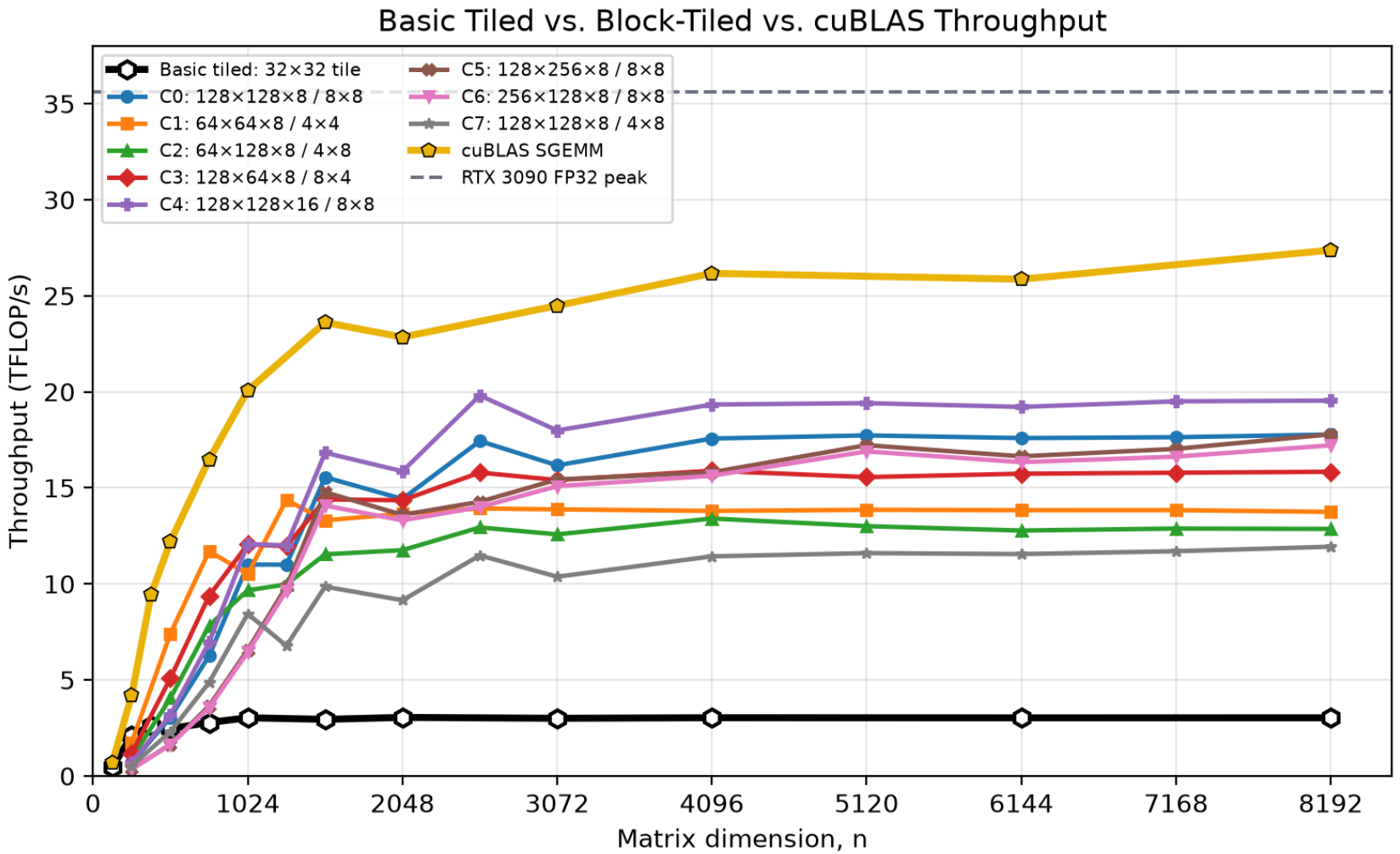
**The remaining gap**

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

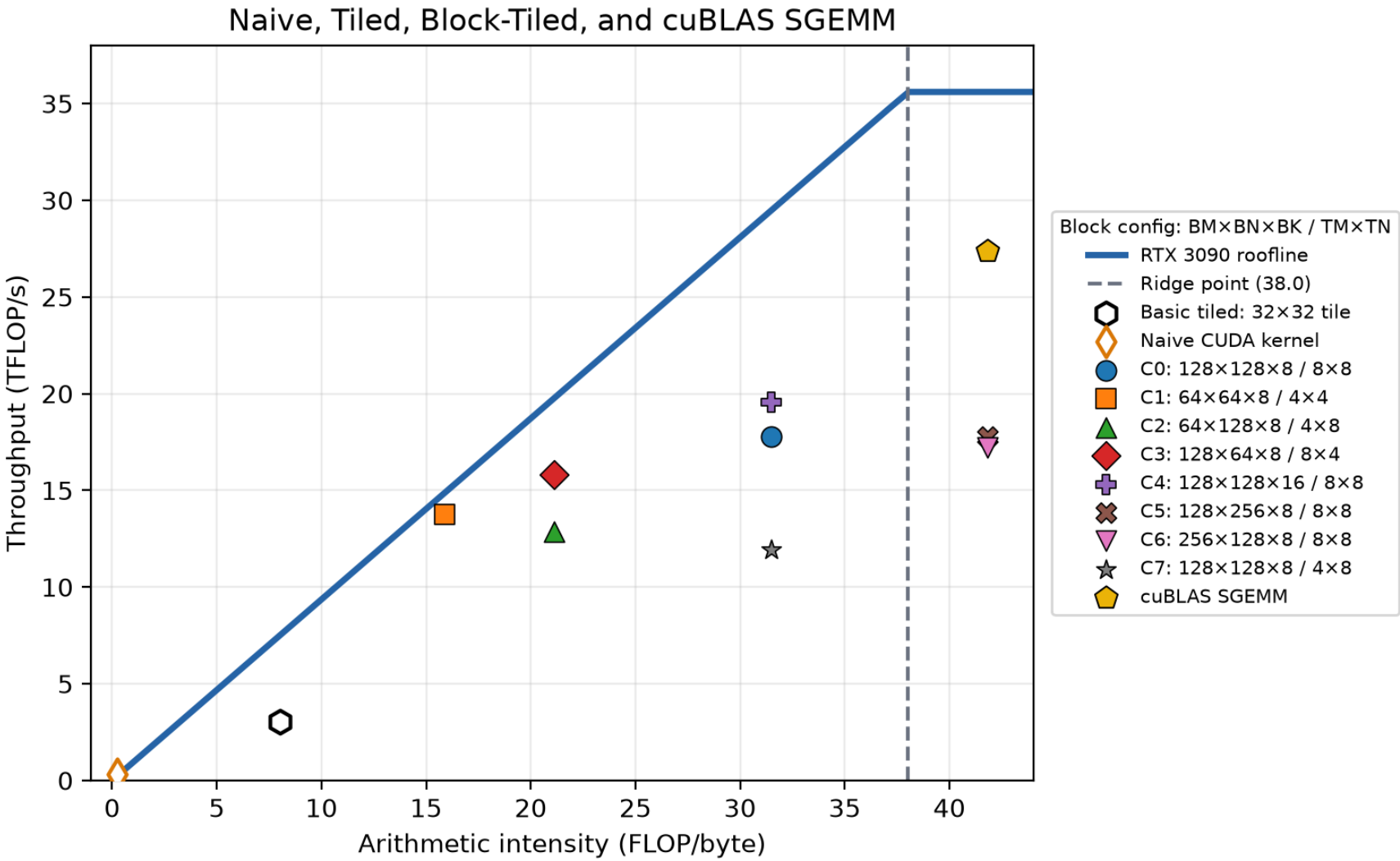
# Basic tiled vs. block-tiled latency vs. cuBLAS



# cuBLAS remains the performance target



# cuBLAS on the roofline




cuBLAS arithmetic intensity is estimated from the profiled $256 \times 128$ output-tile shape.

# We've come a long way

